



(12) **United States Patent**
Sakr et al.

(10) **Patent No.:** **US 12,406,169 B2**
(45) **Date of Patent:** **Sep. 2, 2025**

(54) **OPTIMALLY CLIPPED TENSORS AND VECTORS**

USPC 707/600–899; 706/25
See application file for complete search history.

(71) Applicant: **NVIDIA Corporation**, Santa Clara, CA (US)

(56) **References Cited**

(72) Inventors: **Charbel Sakr**, Mountain View, CA (US); **Steve Haihang Dai**, Union City, CA (US); **Brucek Kurdo Khailany**, Austin, TX (US); **William James Dally**, Incline Village, NV (US); **Rangharajan Venkatesan**, San Jose, CA (US); **Brian Matthew Zimmer**, Berkeley, CA (US)

U.S. PATENT DOCUMENTS

2022/0092426 A1* 3/2022 Pang G06N 3/084

OTHER PUBLICATIONS

Abdolrashidi, A., et al., “Pareto-optimal quantized resnet is mostly 4-bit,” In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 3091-3099, 2021.

(Continued)

(73) Assignee: **NVIDIA Corporation**, Santa Clara, CA (US)

Primary Examiner — Isaac M Woo

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 648 days.

(74) *Attorney, Agent, or Firm* — Leydig, Voit & Mayer, Ltd.

(57) **ABSTRACT**

(21) Appl. No.: **17/814,957**

Quantizing tensors and vectors processed within a neural network reduces power consumption and may accelerate processing. Quantization reduces the number of bits used to represent a value, where decreasing the number of bits used can decrease the accuracy of computations that use the value. Ideally, quantization is performed without reducing accuracy. Quantization-aware training (QAT) is performed by dynamically quantizing tensors (weights and activations) using optimal clipping scalars. “Optimal” in that the mean squared error (MSE) of the quantized operation is minimized and the clipping scalars define the degree or amount of quantization for various tensors of the operation. Conventional techniques that quantize tensors during training suffer from high amounts of noise (error). Other techniques compute the clipping scalars offline through a brute force search to provide high accuracy. In contrast, the optimal clipping scalars can be computed online and provide the same accuracy as the clipping scalars computed offline.

(22) Filed: **Jul. 26, 2022**

(65) **Prior Publication Data**

US 2023/0237308 A1 Jul. 27, 2023

Related U.S. Application Data

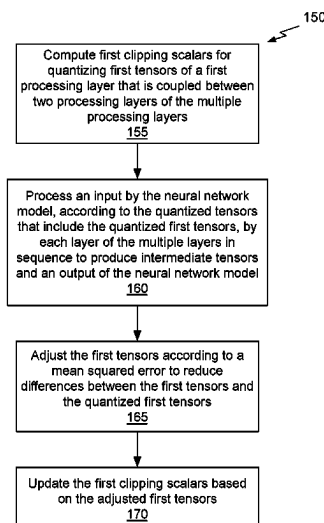
(60) Provisional application No. 63/303,899, filed on Jan. 27, 2022.

(51) **Int. Cl.**
G06F 7/00 (2006.01)
G06N 3/04 (2023.01)
G06N 3/08 (2023.01)

(52) **U.S. Cl.**
CPC **G06N 3/04** (2013.01); **G06N 3/08** (2013.01)

(58) **Field of Classification Search**
CPC G06N 3/04; G06N 3/08

26 Claims, 12 Drawing Sheets

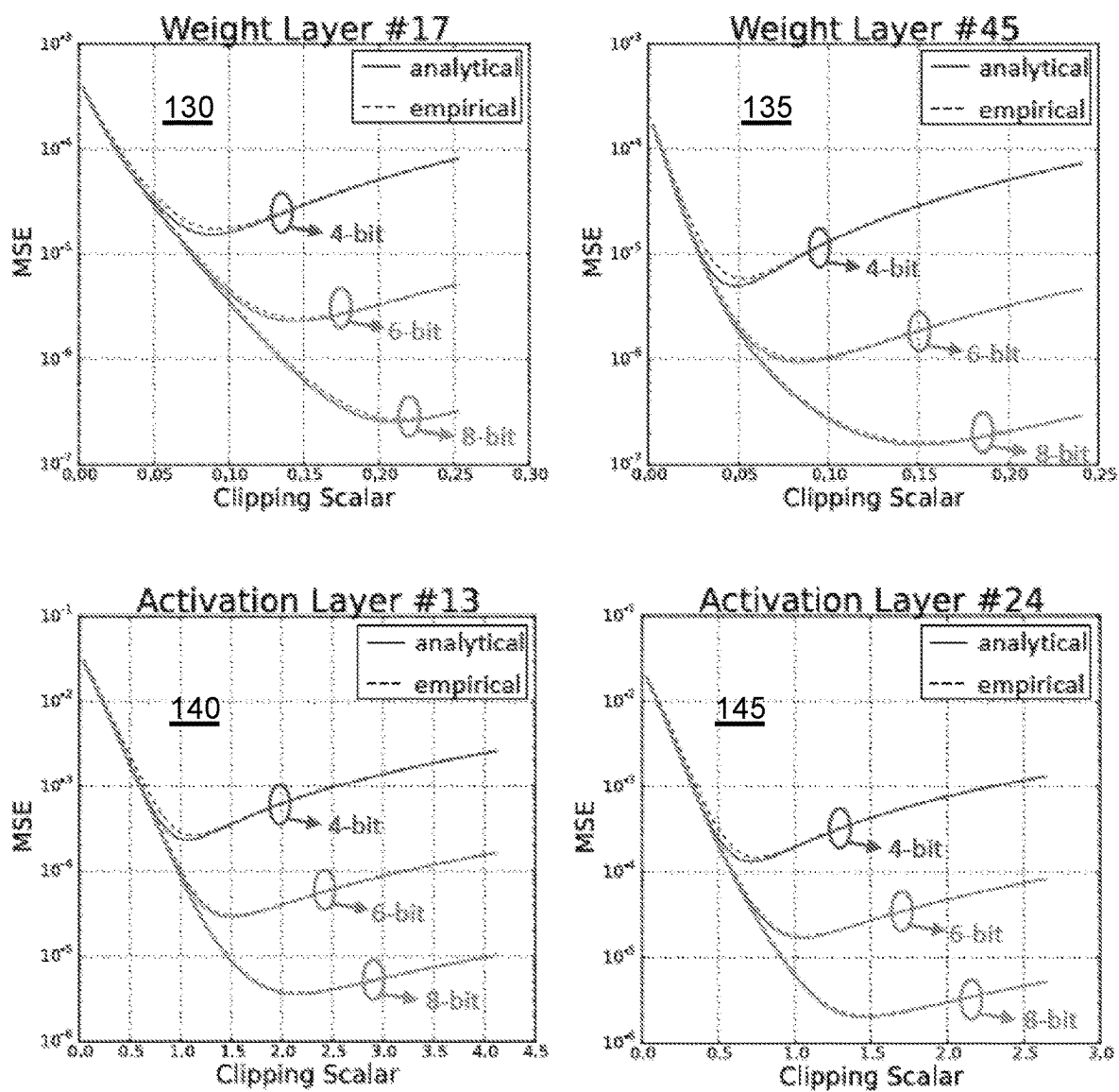


(56)

References Cited**OTHER PUBLICATIONS**

- Bianco, S., et al., "Benchmark analysis of representative deep neural network architectures," *IEEE Access*, 6:64270-64277, 2018.
- Choi, J., et al., "PACT: Parameterized clipping activation for quantized neural networks," *arXiv preprint arXiv:1805.06085*, 2018.
- Choi, Y., et al., "Data-free network quantization with adversarial knowledge distillation," In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 710-711, 2020.
- Courbariaux, M., et al., "BinaryConnect: Training deep neural networks with binary weights during propagations," In *Advances in Neural Information Processing Systems*, pp. 3123-3131, 2015.
- Dai, S., et al., "Vs-Quant: Per-vector scaled quantization for accurate low-precision neural network inference," *Proceedings of Machine Learning and Systems*, 3, 2021.
- Dbouk, H., et al., "DBQ: A differentiable branch quantizer for lightweight deep neural networks," In *European Conference on Computer Vision*, pp. 90-106, Springer, 2020.
- Deng, J., et al., "ImageNet: A large-scale hierarchical image database," In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248-255, IEEE, 2009.
- Devlin, J., et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- Goel, M., et al., "Finite-precision analysis of the pipelined strength-reduced adaptive filter," *Signal Processing, IEEE Transactions on*, 46(6):1763-1769, 1998.
- Gonugondla, S., et al., "Fundamental limits on the precision of in-memory architectures," In *Proceedings of the 39th International Conference on Computer-Aided Design*, pp. 1-9, 2020.
- Gupta, S., et al., "Deep learning with limited numerical precision," In *International Conference on Machine Learning*, pp. 1737-1746, 2015.
- Han, S., et al., "Eie: Efficient inference engine on compressed deep neural network," *ACM SIGARCH Computer Architecture News*, 44(3):243-254, 2016.
- He, K., et al., "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1026-1034, 2015.
- Howard, A., et al., "Searching for mobilenetv3," In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1314-1324, 2019.
- Hubara, I., et al., "Binarized neural networks," In *Advances in Neural Information Processing Systems*, pp. 4107-4115, 2016.
- Jain, S., et al., "BiScaled-DNN: Quantizing long-tailed datastructures with two scale factors for deep neural networks," In *2019 56th ACM/IEEE Design Automation Conference (DAC)*, pp. 1-6, IEEE, 2019.
- Koster, U., et al., "Flexpoint: An adaptive numerical format for efficient training of deep neural networks," In *Advances in Neural Information Processing Systems*, pp. 1740-1750, 2017.
- Krizhevsky, A., et al., "ImageNet classification with deep convolutional neural networks," In *Advances in Neural Information Processing Systems*, pp. 1097-1105, 2012.
- Lecun, Y., et al., "Deep learning," *Nature*, 521(7553):436-444, 2015.
- Lee, E.H., et al., "Lognet: Energy-efficient neural networks using logarithmic computation," In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5900-5904, IEEE, 2017.
- Lin, Y., et al., "PredictiveNet: an energy-efficient convolutional neural network via zero prediction," In *Circuits and Systems (ISCAS)*, 2017 IEEE International Symposium on. IEEE, 2017.
- Lloyd, S., "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, 28(2):129-137, 1982.
- Nagel, M., et al., "Data-free quantization through weight equalization and bias correction," In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1325-1334, 2019.
- Park, E., et al., "Profit: A novel training method for sub-4-bit mobilenet models," In *European Conference on Computer Vision*, pp. 430-446, Springer 2020.
- Paszke, A., et al., "Automatic differentiation in PyTorch," In *NeurIPS Workshop on Automatic Differentiation*, 2017.
- Rajpurkar, P., et al., "SQuAD: 100,000+ questions for machine comprehension of text," In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 2383-2392, 2016.
- Sakr, C., et al., "Per-tensor fixed-point quantization of the back-propagation algorithm," In *7th International Conference on Learning Representations, ICLR 2019*.
- Sakr, C., et al., "Accumulation bit-width scaling for ultra-low precision training of deep networks," In *7th International Conference on Learning Representations, ICLR 2019*.
- Sandler, M., et al., "Mobilenetv2: Inverted residuals and linear bottlenecks," In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4510-4520, 2018.
- Srivastava, N., et al., "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, 15(1):1929-1958, 2014.
- Sun, X., et al., "Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks," In *NeurIPS*, 2019.
- Taigman, Y., et al., "Deepface: Closing the gap to human-level performance in face verification," In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1701-1708, 2014.
- Tambe, T., et al., "Adaptivfloat: A floating-point based data type for resilient deep learning inference," *arXiv preprint arXiv:1909.13271*, 2019.
- Wang, N., et al., "Training deep neural networks with 8-bit floating point numbers," In *Advances in Neural Information Processing Systems*, 2018.
- Widrow, B., et al., "Quantization noise," *Cambridge University Press*, 2:5, 2008.
- Wu, H., et al., "Integer quantization for deep learning inference: Principles and empirical evaluation," *arXiv preprint arXiv:2004.09602*, 2020.
- Zhang, D., et al., "LQ-Nets: Learned quantization for highly accurate and compact deep neural networks," In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 365-382, 2018.
- Zhao, J., et al., "Low-precision training in logarithmic number system using multiplicative weight update," *arXiv preprint arXiv:2106.13914*, 2021.
- Zhou, S., et al., "DoReFa-Net: Training low bandwidth convolutional neural networks with low bandwidth gradients," *arXiv preprint arXiv:1606.06160*, 2016.
- Zhu, Y., et al., "Aligning books and movies: Towards story-like visual explanations by watching movies and reading books," In *Proceedings of the IEEE International conference on computer vision*, pp. 19-27, 2015.

* cited by examiner

*Fig. 1A*

Neural Network

-120

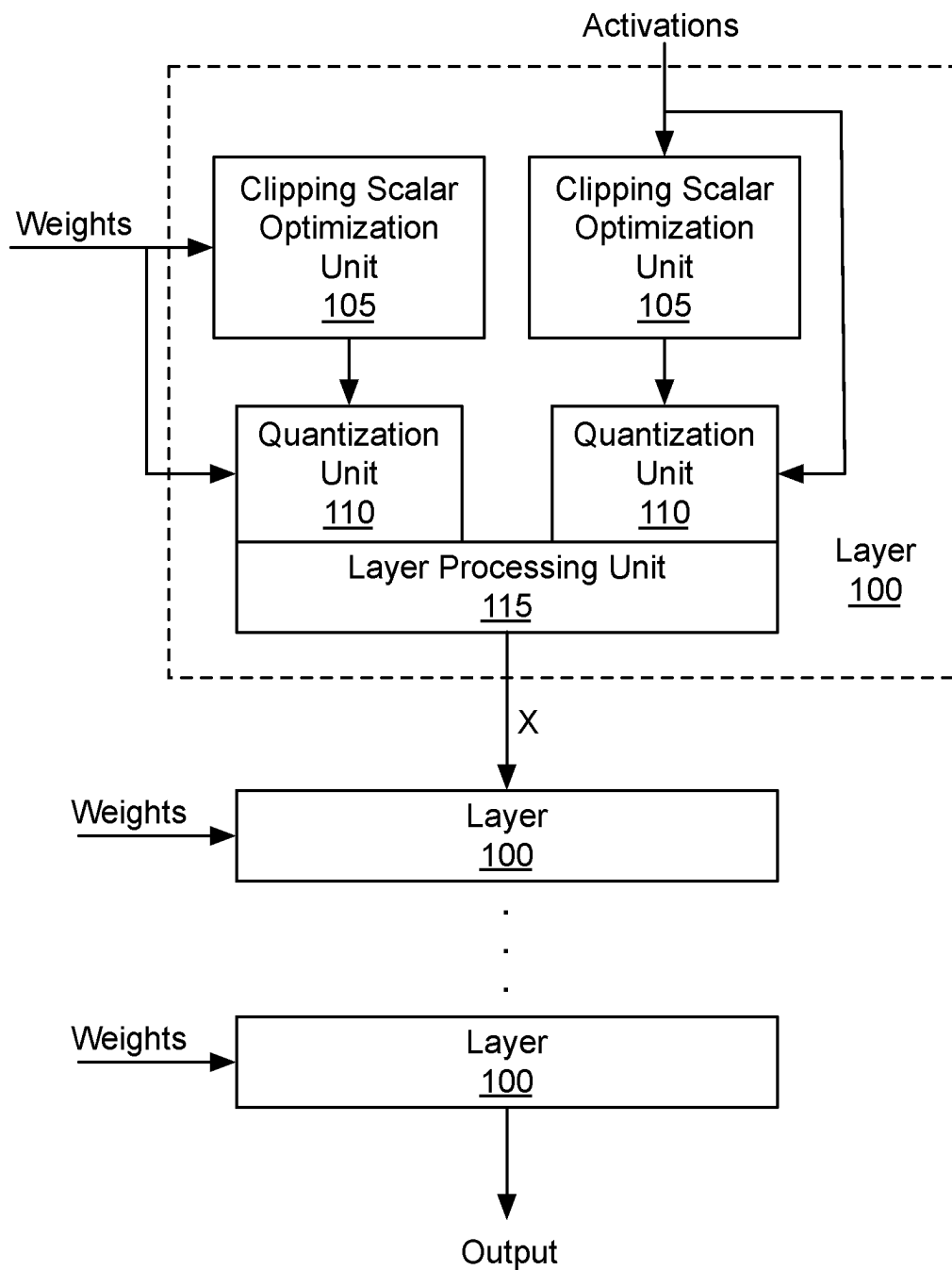
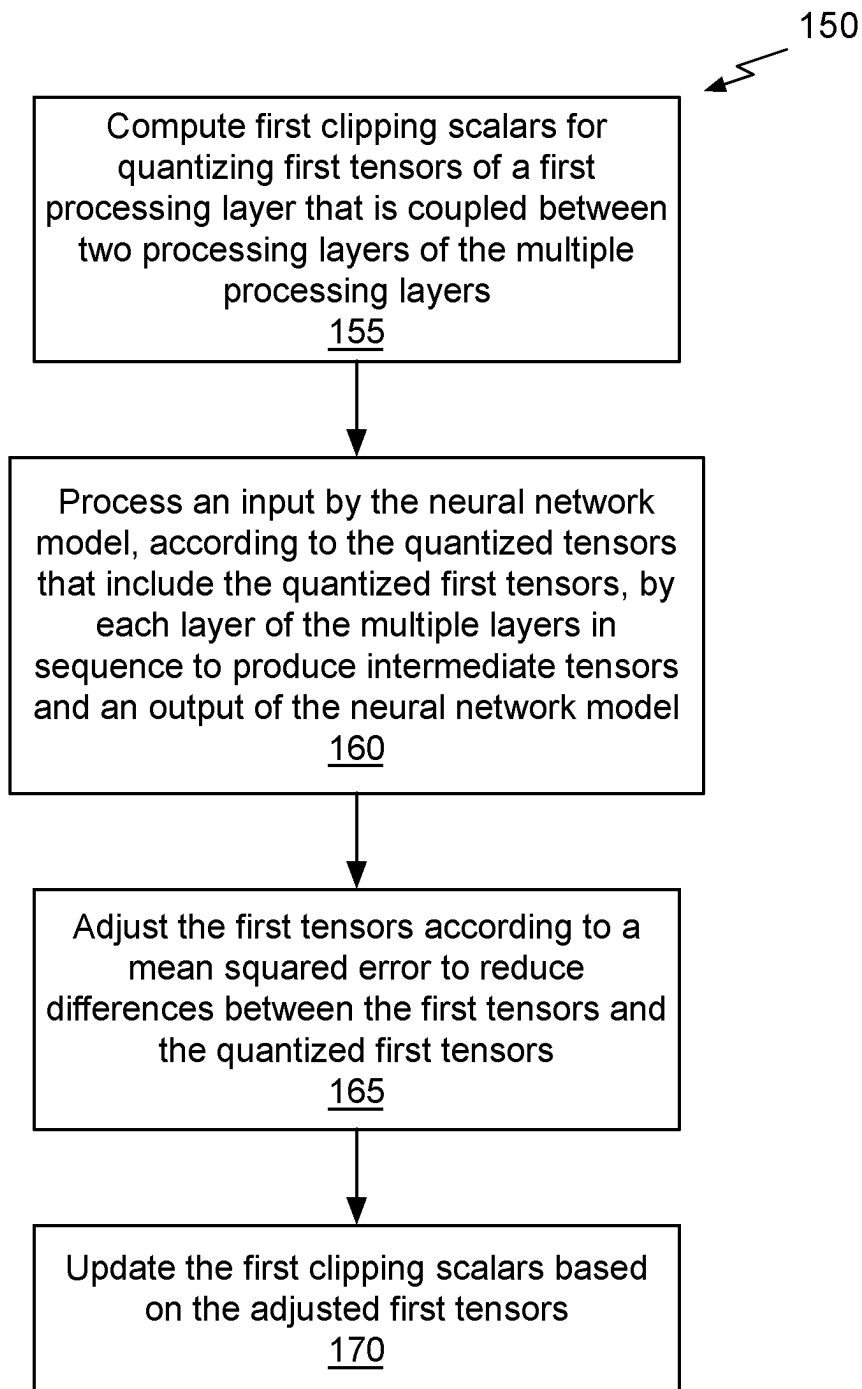
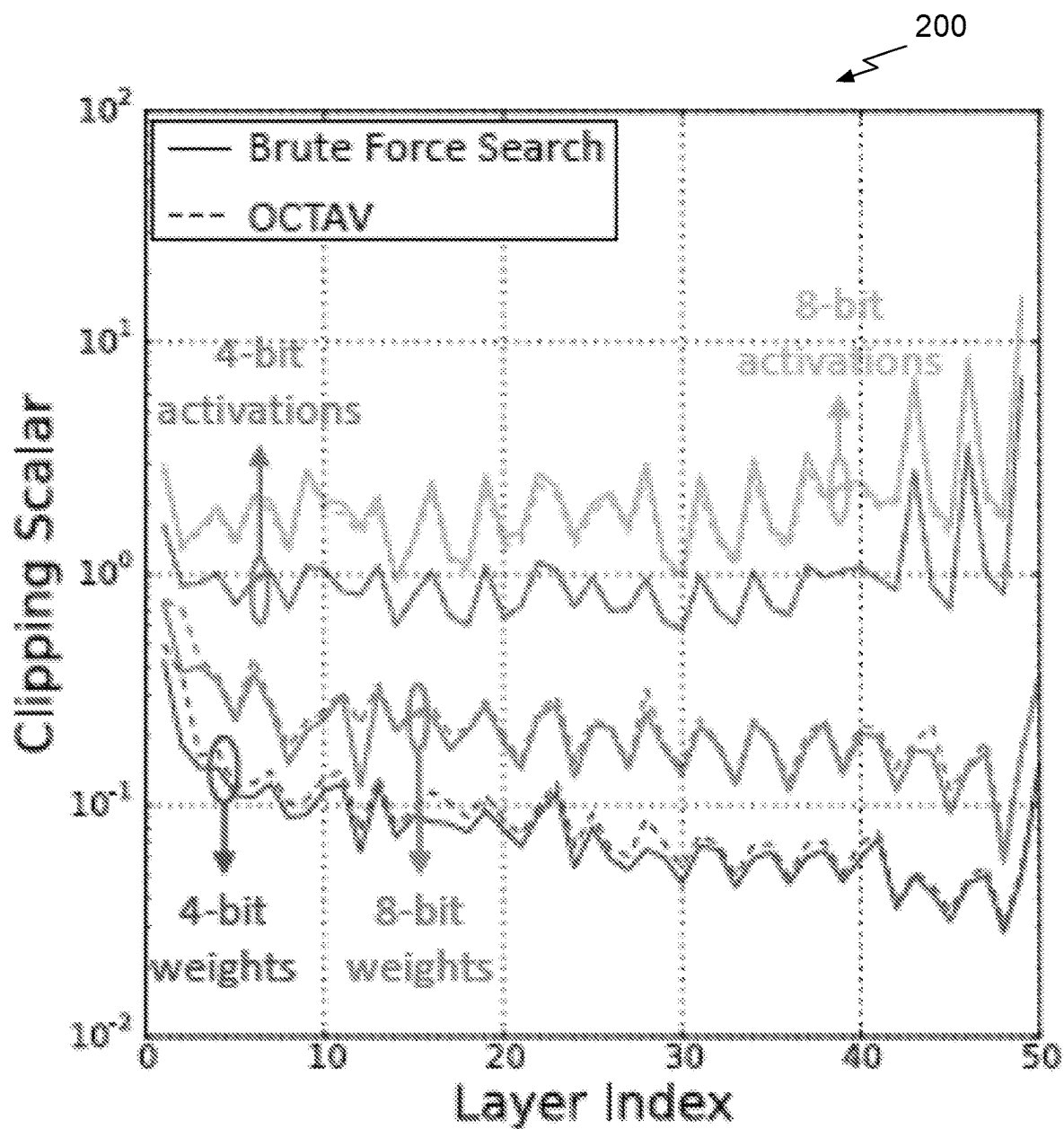


Fig. 1B

*Fig. 1C*

*Fig. 2A*

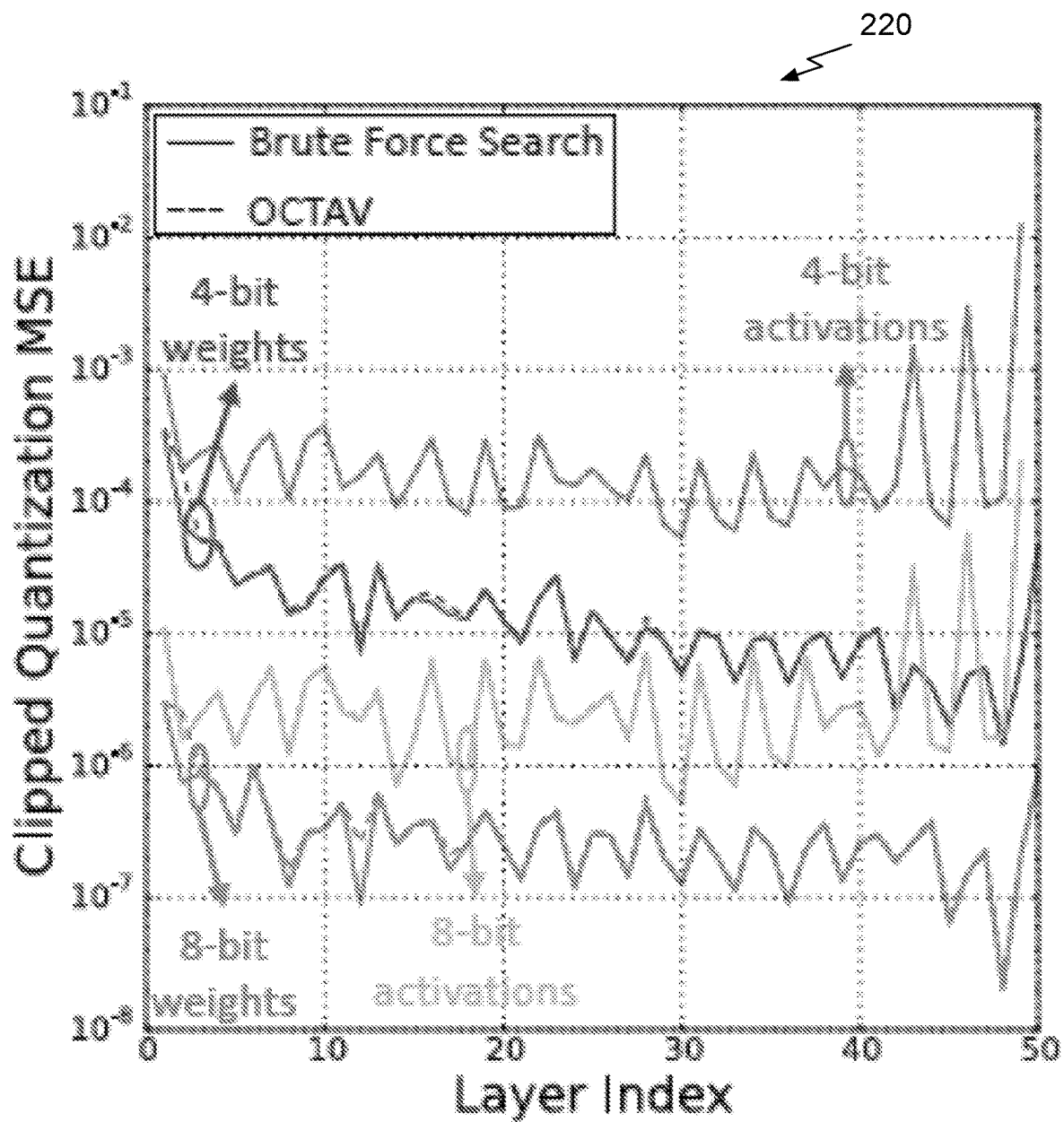
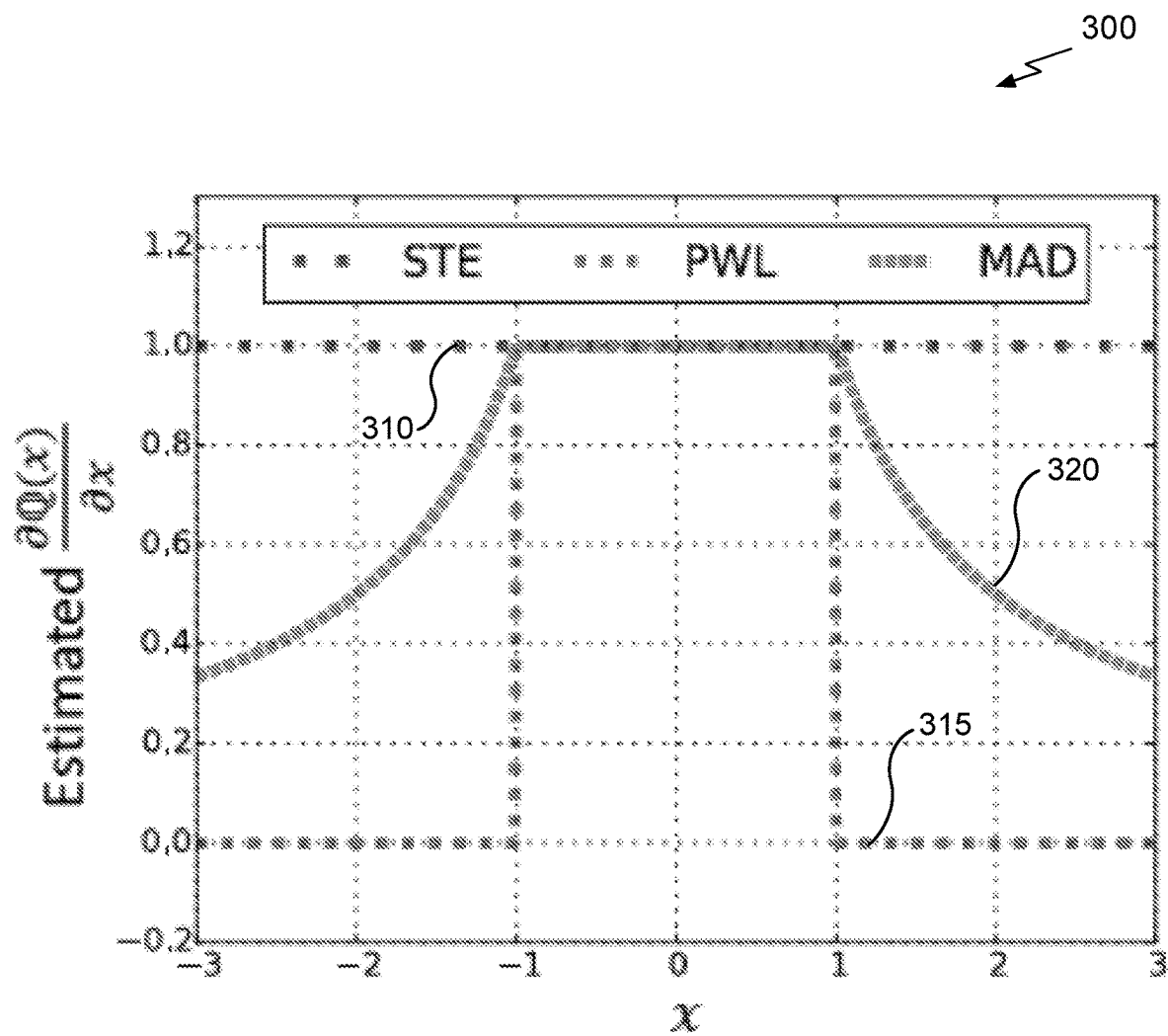
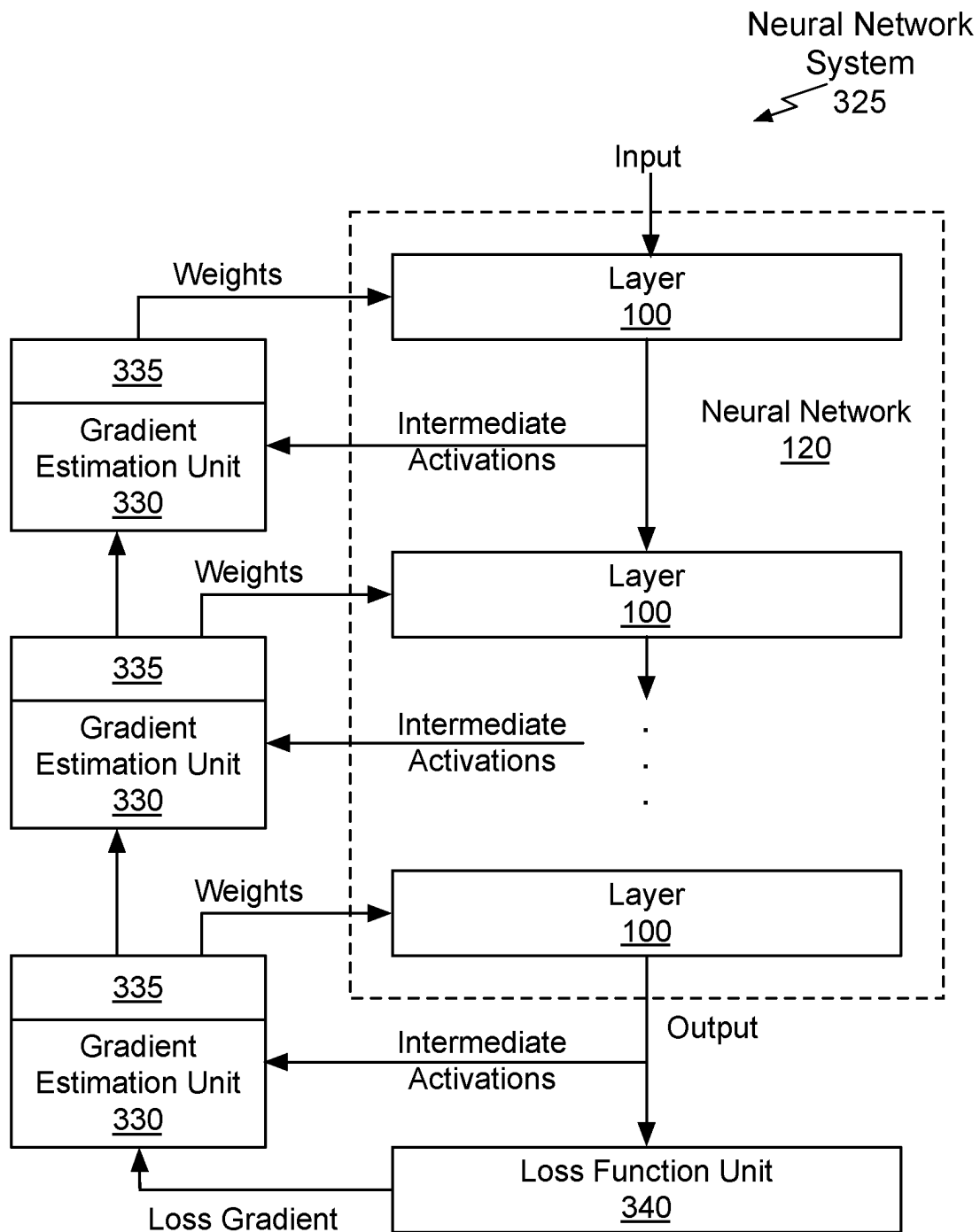
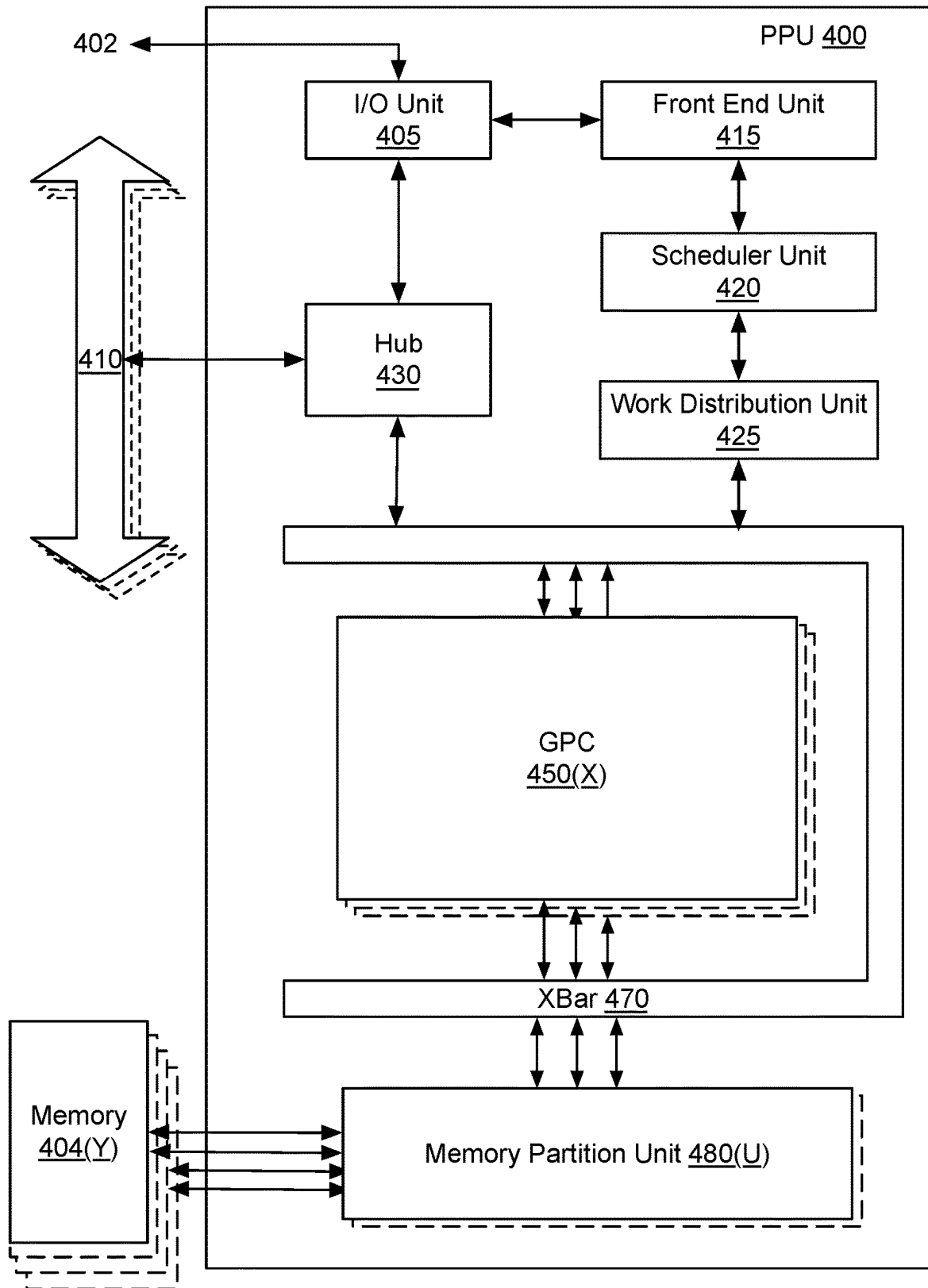
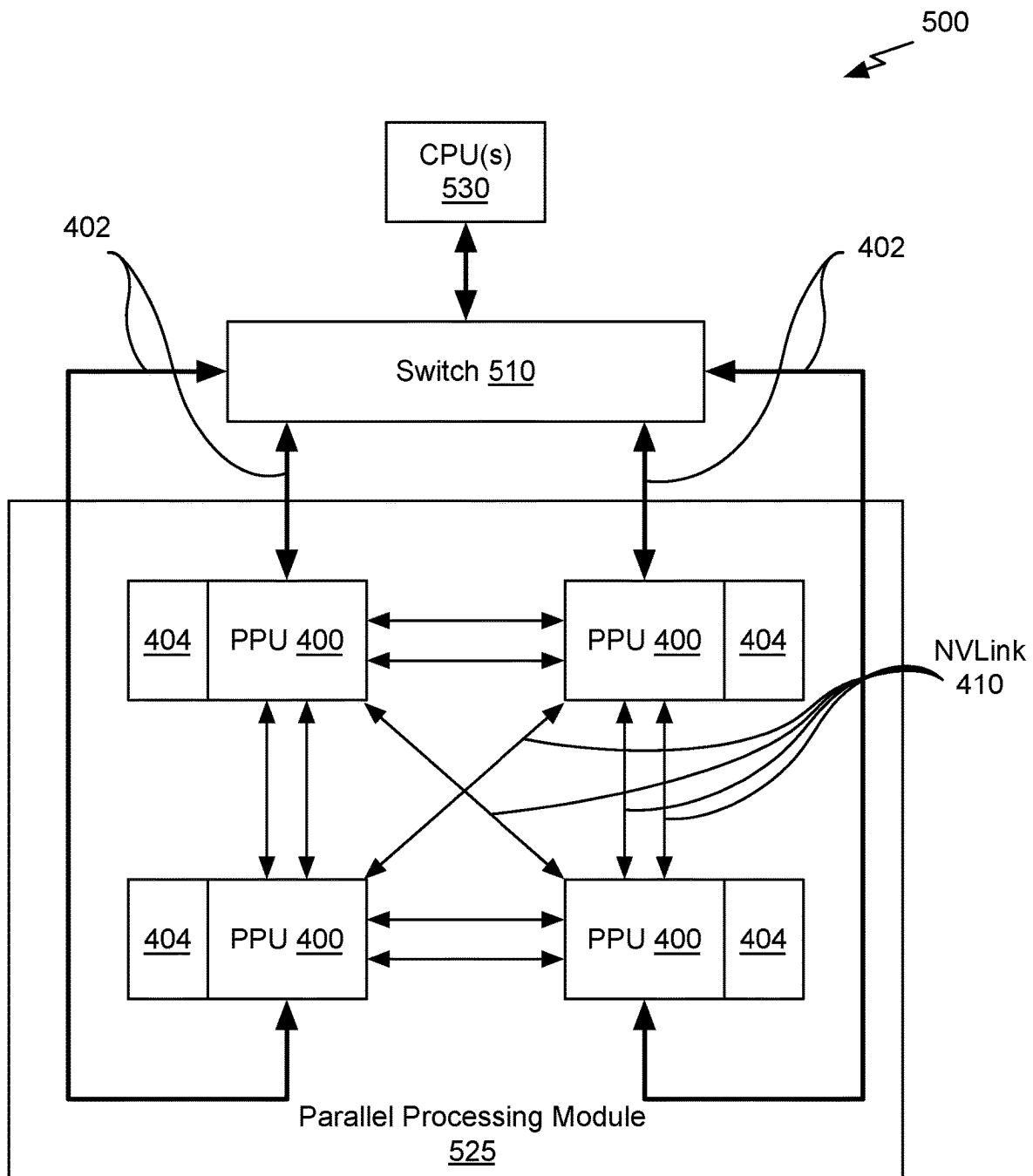


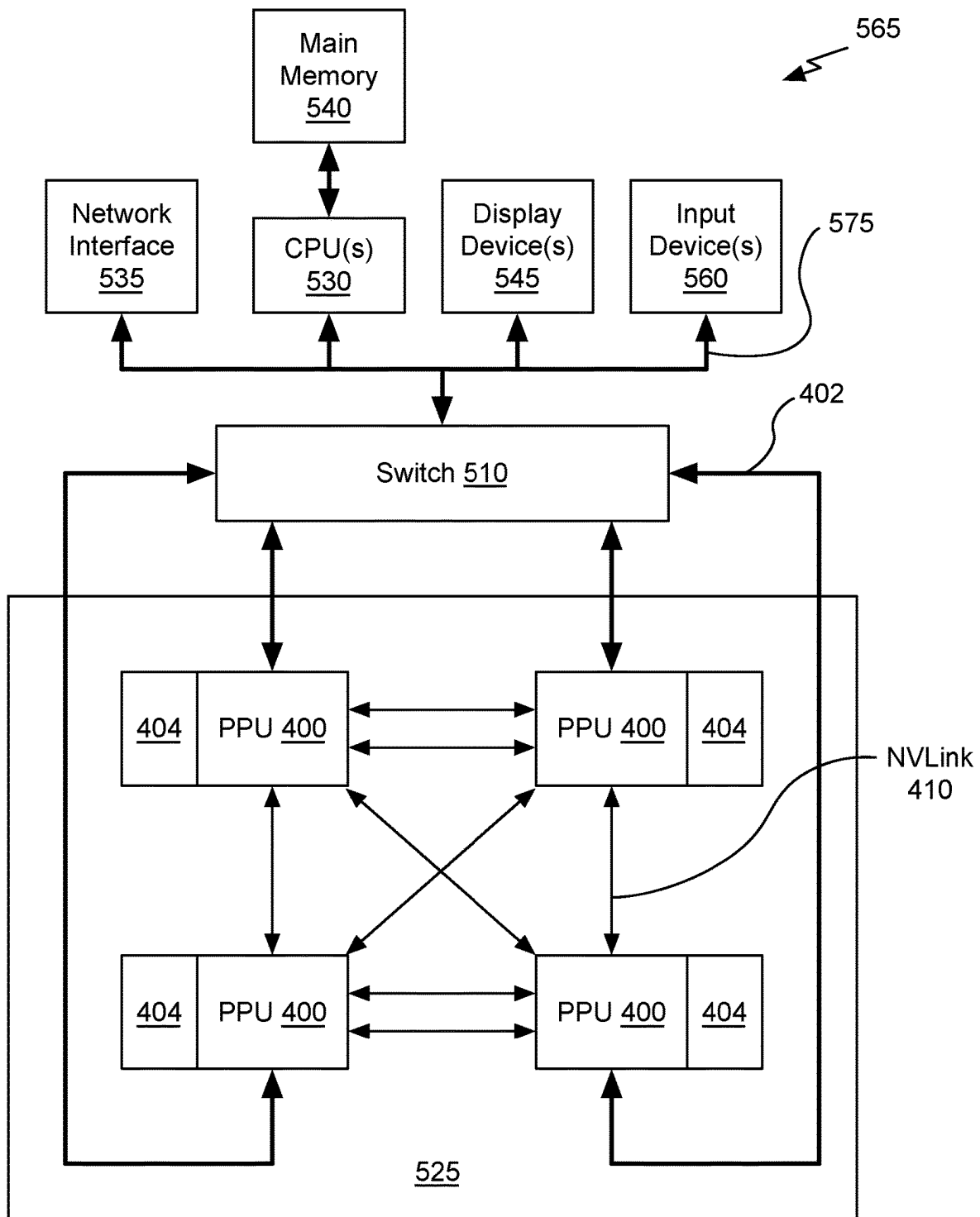
Fig. 2B

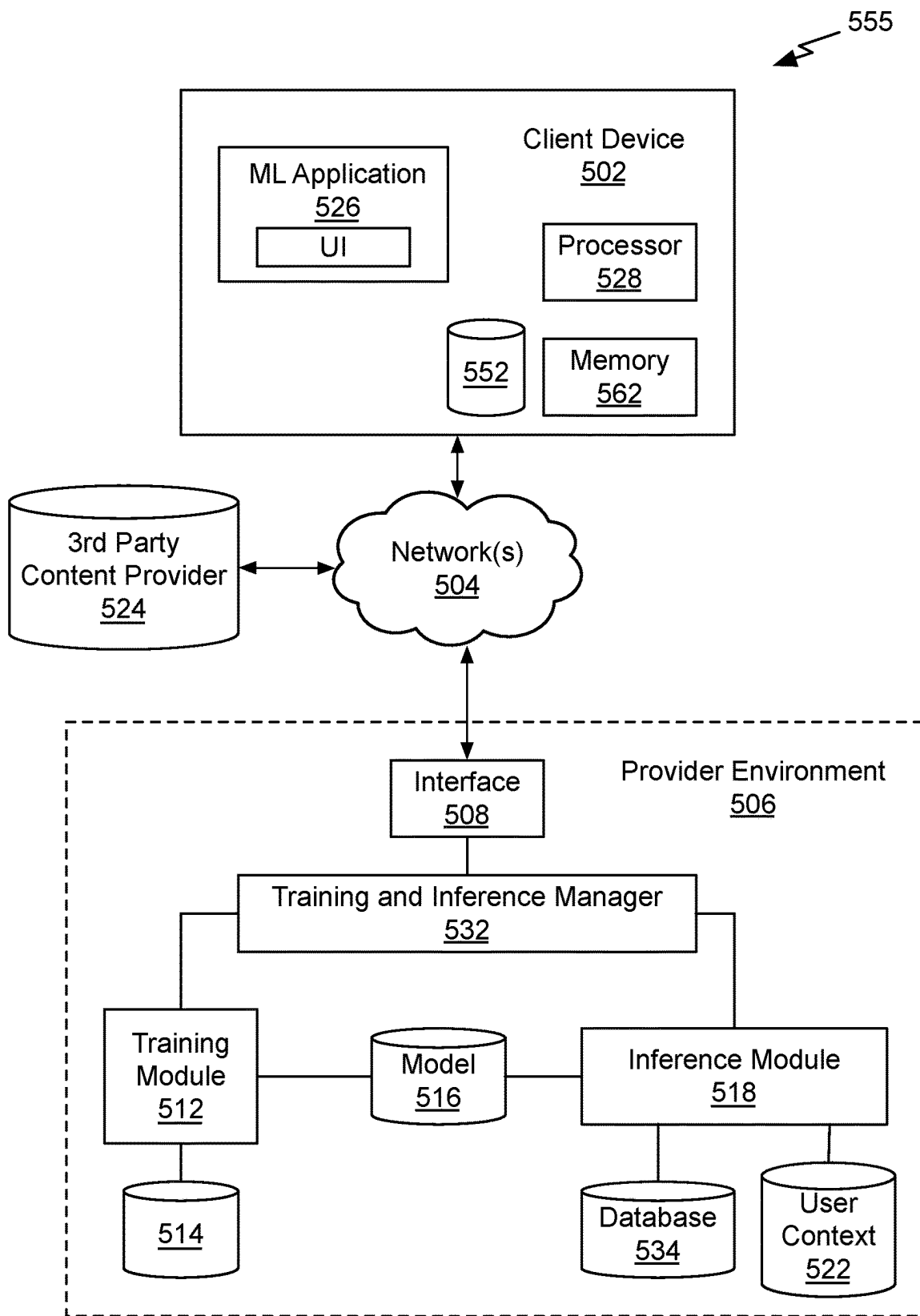
*Fig. 3A*

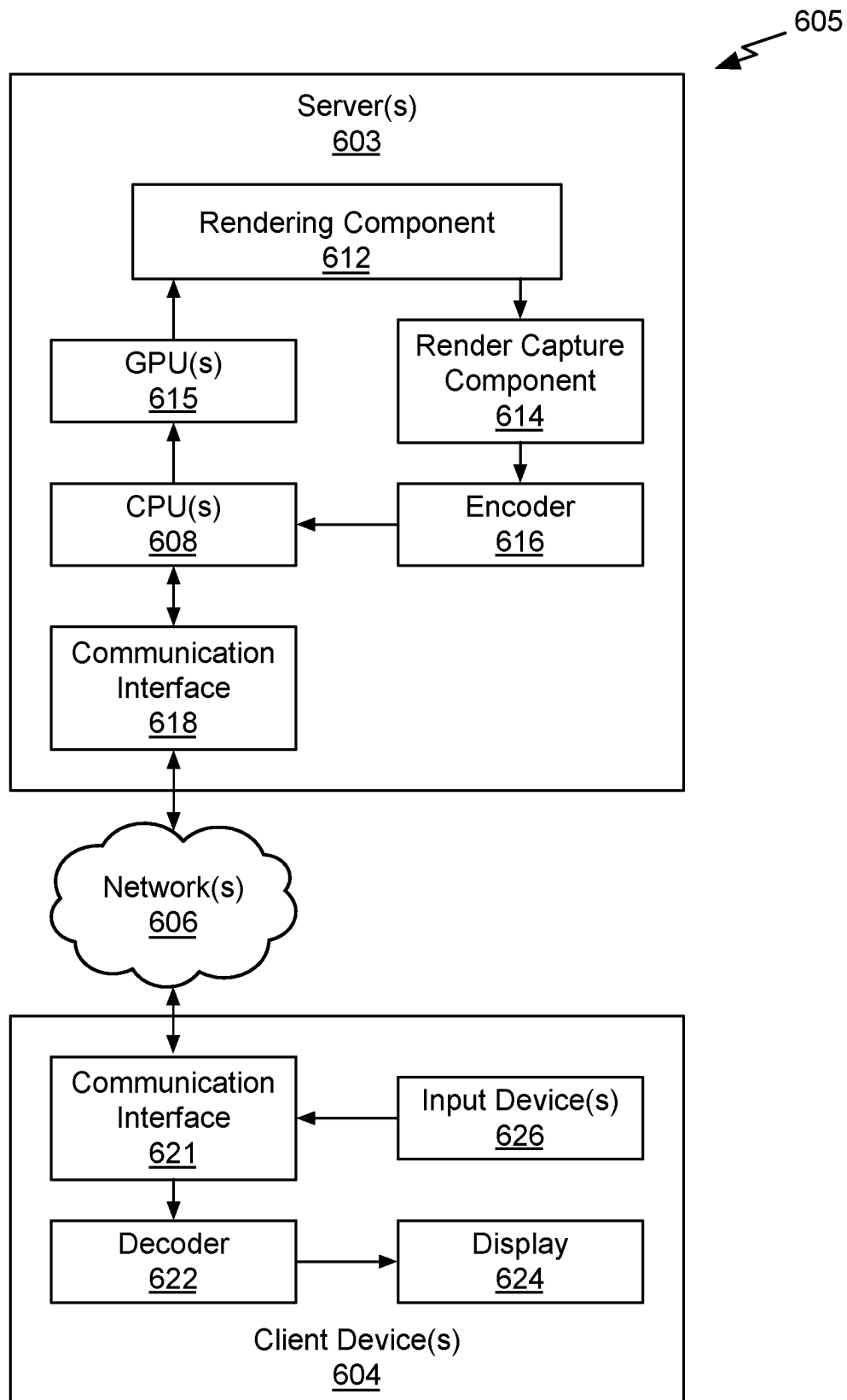
*Fig. 3B*

*Fig. 4*

*Fig. 5A*

*Fig. 5B*

*Fig. 5C*

**Fig. 6**

OPTIMALLY CLIPPED TENSORS AND VECTORS

CLAIM OF PRIORITY

This application claims the benefit of U.S. Provisional Application No. 63/303,899 titled “Optimally Clipped Tensors and Vectors,” filed Jan. 27, 2022, the entire contents of which is incorporated herein by reference.

BACKGROUND

Quantizing tensors and vectors processed within a neural network reduces power consumption and may accelerate processing. Quantization reduces the number of bits used to represent a value, where decreasing the number of bits used can decrease the accuracy of computations that use the value. Quantization is a widely adopted technique used to improve the efficiency of deep learning acceleration. However, maintaining task-level accuracy at very low precision, e.g., sub-8-bit, is very challenging due to the presence of excess quantization noise. Thus, there is a need for improving the accuracy of deep learning in the presence of limited numerical precision.

SUMMARY

Systems and methods are disclosed that quantize tensors and vectors. Embodiments of the present disclosure relate to optimally clipped tensors and vectors. Ideally, quantization is performed without reducing accuracy. Quantization-aware training (QAT) is performed by dynamically quantizing tensors (weights and activations) using optimal clipping scalars. “Optimal” in that the mean squared error (MSE) of the quantized operation is minimized and the clipping scalars define the degree or amount of quantization for various tensors of the operation. Conventional techniques that quantize tensors during training suffer from high amounts of noise (error). Other conventional techniques compute the clipping scalars offline through a brute force search to provide high accuracy. In contrast to conventional techniques, the optimal clipping scalars can be computed online and provide the same accuracy as the clipping scalars computed offline.

In an embodiment, the method for quantizing tensors of a neural network model comprising multiple processing layers during training includes computing first clipping scalars for quantizing first tensors of a first processing layer that is coupled between two processing layers of the multiple processing layers. An input is processed by the neural network model according to the quantized tensors by each layer of the multiple layers in sequence to produce intermediate tensors and an output of the neural network model, the first tensors are adjusted according to a mean squared error to reduce differences between the first tensors and the quantized tensors, and the first clipping scalars are updated based on the adjusted first tensors.

BRIEF DESCRIPTION OF THE DRAWINGS

The present systems and methods for optimally clipped tensors and vectors are described in detail below with reference to the attached drawing figures, wherein:

FIG. 1A illustrates quantization MSE as a function of clipping scalar, in accordance with an embodiment.

FIG. 1B illustrates a block diagram of an example neural network suitable for use in implementing some embodiments of the present disclosure.

FIG. 1C illustrates a flowchart of a method for quantizing tensors of a neural network model, in accordance with an embodiment.

FIG. 2A illustrates clipping scalars determined by brute force search compared with the computed optimally clipped scalars, in accordance with an embodiment.

FIG. 2B illustrates clipping quantization MSE determined by brute force search compared with the computed optimally clipped scalars, in accordance with an embodiment.

FIG. 3A illustrates gradient estimation functions for a clipped quantization operation, in accordance with an embodiment.

FIG. 3B illustrates a block diagram of an example neural network suitable for use in implementing some embodiments of the present disclosure.

FIG. 4 illustrates an example parallel processing unit suitable for use in implementing some embodiments of the present disclosure.

FIG. 5A is a conceptual diagram of a processing system implemented using the PPU of FIG. 4, suitable for use in implementing some embodiments of the present disclosure.

FIG. 5B illustrates an exemplary system in which the various architecture and/or functionality of the various previous embodiments may be implemented.

FIG. 5C illustrates components of an exemplary system that can be used to train and utilize machine learning, in at least one embodiment.

FIG. 6 illustrates an exemplary streaming system suitable for use in implementing some embodiments of the present disclosure.

DETAILED DESCRIPTION

Systems and methods are disclosed related to optimally clipped tensors and vectors. Quantization-aware training (QAT) may be performed by dynamically quantizing tensors (weights and activations) using optimal clipping scalars. The clipping scalars are determined and “optimal” in that the mean squared error (MSE) of the quantized operation is minimized and the clipping scalars define the degree or amount of quantization for various tensors of the operation. In an embodiment, the clipping scalars are initialized to random values. In an embodiment, the MSE is minimized to determine the optimal clipping scalars without explicit computation of the MSE. Instead, in an embodiment, the optimal clipping scalars are computed recursively during forward propagation (inference) using the Newton-Raphson algorithm. In an embodiment, the optimal clipping scalars converge in only 10 iterations. The optimal clipping scalars may be dynamically or statically updated during online inference or supervised training. Dynamic quantization is useful for setups with time-varying tensor statistics, such as training-from-scratch and fine-tuning. Alternatively, static quantization is best suited for application on a pretrained setup or when tensor statistics minimally change over time, e.g., in a short retraining or fine-tuning setup. An advantage of static quantization is its conceptual simplicity; once a good calibration strategy is identified, a QAT routine can be readily and reproducibly implemented.

There are three main use cases for QAT: training-from-scratch where the starting point is a randomly initialized neural network; retraining where a pretrained neural network model is quantized and retrained for a short time on the

same dataset; and fine-tuning where the starting point is a neural network model pretrained on one dataset and trained on another.

Some conventional techniques attempt to improve training accuracy in the presence of quantization through extensive hyperparameter tuning. Another conventional technique referred to as max-scaling matches the largest quantized representation to the largest value in the set of elements (tensor or vector) to be quantized. An advantage of max-scaling is that it is well-defined. Indeed, max-scaled QAT can be implemented using the same training recipe as a full precision baseline, simply by inserting the quantization operations where appropriate. Thus, there are no hyperparameters required, and results can be readily reproduced. Unfortunately, max-scaling incurs large amounts of quantization noise, harming accuracy due to large step-sizes. Conventional techniques that provide high accuracy compute the clipping scalars offline through a brute force search. In contrast, the optimal clipping scalars can be computed online and provide accuracy similar to the clipping scalars computed offline via brute force search.

Quantization accuracy can be improved using data clipping that relies on an effective calibration strategy. The calibration problem can be solved in a mathematically sound way, with optimality guarantees. Considering some data x derived from a distribution $f_X(\cdot)$, a B -bit quantization is defined as the process of mapping x to one of 2^B predefined levels $\{r_i\}_{i=1}^{2^B}$. The quantized data is obtained as:

$$Q(x) = \underset{r_i}{\operatorname{argmin}} |x - r_i|.$$

The choice of $\{r_i\}_{i=1}^{2^B}$ is crucial in setting the fidelity of quantization. The quantization noise J may be metricized via the mean squared error (MSE):

$$J = E[(Q(X) - X)^2]. \quad \text{Eq. (1)}$$

For unconstrained quantization, the metric can be minimized using the Lloyd-Max algorithm.

Among uniformly constrained quantizers, max-scaled quantization assumes there exists a scalar $s_{\max}$ such that $f_X(x) = 0$ for $|x| > s_{\max}$. In practice, $s_{\max}$ can be the largest available element in absolute value. The max-scaled quantizer, using a number of bits B , assigns the levels $\{r_i\}_{i=1}^{2^B}$ as an arithmetic progression on $[-s_{\max}, s_{\max}]$. Thus, the max-scaled quantization operation is given by:

$$Q(x) = s_{\max} \cdot 2^{1-B} \cdot \operatorname{round}(x \cdot 2^{B-1} / s_{\max}) \quad \text{Eq. (2)}$$

with the rounding operation being applied on integers. The max-scaled quantizer has been extensively studied in signal processing and machine learning and its MSE, derived using an additive model of quantization noise, is given by

$$J = s_{\max}^2 \frac{4^{-B}}{3}.$$

Quantization accuracy of max-scaling, and uniform quantization in general, can be improved using data clipping. Specifically, a narrower quantization interval $[-s, s]$ is used, with the clipping scalar $s < s_{\max}$ and the quantization operation given by:

$$Q(x) = \operatorname{clip}\left(s \cdot 2^{1-B} \cdot \operatorname{round}\left(\frac{x \cdot 2^{B-1}}{s}\right), -s, s\right) \quad \text{Eq. (3)}$$

$$= \begin{cases} -s & \text{if } x < -s \\ \operatorname{round}\left(\frac{x \cdot 2^{B-1}}{s}\right) & \text{if } x \in [-s, s] \\ s & \text{if } x > s \end{cases}$$

For a given number of bits B and data distribution, the MSE in equation (1) is therefore a function of the clipping scalar s , and an analytical expression may be derived for $J(s)$.

$$J(s) = \frac{4^{-B}}{3} s^2 \int_0^s f_{|X|}(x) dx + \int_s^\infty (s-x)^2 f_{|X|}(x) dx \quad \text{Eq. (4)}$$

where $f_{|X|}(\cdot)$ is the distribution of the absolute value of the data. Equation (4) is obtained by evaluating equation (1) using the law of total expectation; the aforementioned additive noise model is assumed on the discretization interval $[-s, s]$ and the definition of MSE is used when clipping occurs.

FIG. 1A illustrates quantization MSE as a function of clipping scalar, in accordance with an embodiment. Quantization MSE varies as a function of clipping scalar s for two arbitrary weight and activation layers in a pretrained ResNet-50 neural network model. Graphs 130, 135, 140, and 145 correspond to weight layer 17, weight layer 45, activation layer 13, and activation layer 24, respectively. Activation data is obtained by sampling a random input batch from the training set. Solid lines are obtained by evaluating the derived analytical expression for the MSE (equation 4), which requires histograms and numerical integration. Dashed lines are obtained by empirically evaluating the MSE, i.e., quantizing each tensor element and averaging resulting squared errors. Several important observations can be made based on the graphs 130, 135, 140, and 145.

First, the analytical formula for the MSE closely matches the empirical MSE. Second, there exists an optimal clipping scalar s^* minimizing the quantization MSE. The optimum clipping scalar balances the trade-off between discretization and clipping noise. When $s < s^*$, excess clipping leads to an increase in J despite smaller discretization noise. Conversely, when $s > s^*$, clipping is minimal, but the larger quantization step size causes an increase in discretization noise and J . Third, the optimal clipping scalar s^* is a function of both data distribution $f_{|X|}(\cdot)$ and number of bits B . The dependence on $f_{|X|}(\cdot)$ is identified by virtue of s^* being different for different layers (e.g., when $B=4$, $s^* \approx 0.1$ and 0.05 for weight layers #17 and #45, respectively). The dependence on B is identified because s^* varies when B changes and the data is unchanged (e.g., for activation layer #13, $s^* \approx 1.0$ and 2.0 for $B=4$ and 8 , respectively).

Finding s^* can be done offline through a brute force search, i.e., sweeping the value of s . However, the brute force search is highly time-consuming and difficult to implement dynamically. The analytical expression of the MSE requires histograms to estimate the data distribution and evaluation of the analytical expression of the MSE uses numerical integration. Similarly, an empirical evaluation requires successive rounding and reduction operations on large tensors. As described further herein, the optimal scalar s^* may be computed online with reduced quantization noise compared with existing online quantization techniques and that provide the same accuracy as the clipping scalars

computed offline. The online computation enables quantization-aware training (QAT) where forward tensors are quantized to low bitwidths. Specifically, training with the least amount of quantization noise to estimate the achievable accuracy of a learning algorithm is possible.

FIG. 1B illustrates a block diagram of an example neural network 120 suitable for use in implementing some embodiments of the present disclosure. It should be understood that this and other arrangements described herein are set forth only as examples. Other arrangements and elements (e.g., machines, interfaces, functions, orders, groupings of functions, etc.) may be used in addition to or instead of those shown, and some elements may be omitted altogether. Further, many of the elements described herein are functional entities that may be implemented as discrete or distributed components or in conjunction with other components, and in any suitable combination and location. Various functions described herein as being performed by entities may be carried out by hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. Furthermore, persons of ordinary skill in the art will understand that any system that performs the operations of the neural network 120 is within the scope and spirit of embodiments of the present disclosure.

The neural network 120 includes a sequence of multiple processing layers 100. Each layer 100 applies weights to activation inputs and produces an output (X) that comprises activation inputs to a subsequent layer 100. The output of the final layer 100 is the output of the neural network 120. The weights and/or activation values are quantized for input to each layer 100. Each layer 100 includes a layer processing unit 115 and quantizing logic that includes a first clipping scalar and optimization unit 105 and quantization unit 110 pair for quantizing the weights and a second clipping scalar and optimization unit 105 and quantization unit 110 pair for quantizing the activations. The clipping scalar optimization units 105 each receive a reference that is the unquantized weights or activations (i.e., full precision weights and activations) and compute optimal clipping scalar(s). The weights and activations are each quantized by the quantization units 110 according to the optimal clipping scalars computed by the respective optimal clipping scalar optimization unit 105. The clipping scalars may be equal for all of the layers 100 or vary for one or more of the layers 100. Specific optimal clipping scalars may be computed for each tensor, vector, and/or channel of the weight and/or activation for each layer 100. A vector comprises one or more elements within a single dimension of a multi-dimensional tensor (or kernel). The quantization unit 110 receives the optimal clipping scalar(s) computed by the clipping scalar optimization unit 105 and either the weight or activation that is quantized according to the optimal clipping scalar(s). The quantized weight and activation are input to the layer processing unit 115.

In an embodiment, the optimal clipping scalars are recursively computed during forward propagation using the Newton-Raphson algorithm. In an embodiment, the clipping scalars converge to the optimal values in only ten iterations. In an embodiment, the clipping scalars are initialized to random values. In an embodiment, a mean squared error (MSE) is minimized to determine the optimal clipping scalars without explicit computation of the MSE.

Given a data distribution $f_X(\cdot)$ the clipping scalar s^* that minimizes the clipped quantization MSE in equation (4) can be found by assigning a random guess s_1 and recursively computing $\{s_n\}_{n>1}$ until convergence using

$$s_{n+1} = \frac{\mathbb{E}[|X| \cdot \mathbb{1}_{\{|X|>s_n\}}]}{\frac{4^{-B}}{3} \mathbb{E}[\mathbb{1}_{\{|X|\leq s_n\}}] + \mathbb{E}[\mathbb{1}_{\{|X|>s_n\}}]}. \quad \text{Eq. (5)}$$

The Newton-Raphson algorithm may be used to compute

$$s_{n+1} = s_n - \frac{J'(s_n)}{J''(s_n)}.$$

Evaluating first and second derivatives of $J(s)$ leads to the recursive expression in equation (5) that applies to an arbitrary distribution.

For tensor and vector quantization, the optimal clipping scalar s^* that minimizes the clipped quantization MSE in a tensor or vector $\vec{t}$, can be found as follows. Specifically, for $\vec{t}$, a random guess s_1 is assigned and $\{s_n\}_{n>1}$ is recursively computed until convergence using:

$$s_{n+1} = \frac{\sum_{x \in \vec{t}} [|x| \cdot \mathbb{1}_{\{|x|>s_n\}}]}{\frac{4^{-B}}{3} \sum_{x \in \vec{t}} [\mathbb{1}_{\{0<|x|\leq s_n\}}] + \sum_{x \in \vec{t}} [\mathbb{1}_{\{|x|>s_n\}}]}. \quad \text{Eq. (6)}$$

The algorithm defined by equation (6), which is referred to as Optimally Clipped Tensors And Vectors (OCTAV), converges to the optimal clipping scalar s^* . The OCTAV algorithm is guaranteed to converge to the global optimum of the convex MSE $J(s)$. A trade-off between clipping and quantization noise leads to a convexity that can be verified because the second derivative $J''(s)$ is positive.

Compared with equation (5), the empirical distribution of the data inside $\vec{t}$ is used in lieu of the abstract $f_X(\cdot)$, to derive equation (6). Thus, expectations in equation (5) are replaced by average summations in equation (6). Numerator and denominator are both multiplied by the number of elements in $\vec{t}$, suppressing the need for division. As zeros can be represented using integer quantization, zero elements in $\vec{t}$ are excluded from the distribution (see first term in the denominator). This is done to prevent an over-estimation of the total quantization noise for very sparse tensors. In an embodiment, the clipping scalar optimization unit 105 computes the optimal clipping scalars using equation (6).

OCTAV converges quickly and is insensitive to the choice of initial guess. Specifically, for various choices of s_1 , consistent convergence to the same solution is achieved. Computationally, each iteration of the OCTAV algorithm can be implemented using operations that may be performed quickly and efficiently. Indeed, the only vector/tensor operations required by equation (5) are the indicator function (realized via datatype casting), and element-wise absolute values, multiplications, and comparisons. Afterwards, sum reductions are performed, and only residual scalar operations remain, including one division.

More illustrative information will now be set forth regarding various optional architectures and features with which the foregoing framework may be implemented, per the desires of the user. It should be strongly noted that the following information is set forth for illustrative purposes and should not be construed as limiting in any manner. Any of the following features may be optionally incorporated with or without the exclusion of other features described.

FIG. 1C illustrates a flowchart of a method 150 for quantizing tensors of a neural network model comprising multiple processing layers, in accordance with an embodiment. Each block of method 150, described herein, comprises a computing process that may be performed using any combination of hardware, firmware, and/or software. For instance, various functions may be carried out by a processor executing instructions stored in memory. The method may also be embodied as computer-usable instructions stored on computer storage media. The method may be provided by a standalone application, a service or hosted service (standalone or in combination with another hosted service), or a plug-in to another product, to name a few. In addition, method 150 is described, by way of example, with respect to the neural network 120 of FIG. 1B. However, this method may additionally or alternatively be executed by any one system, or any combination of systems, including, but not limited to, those described herein. Furthermore, persons of ordinary skill in the art will understand that any system that performs method 150 is within the scope and spirit of embodiments of the present disclosure.

At step 155, first clipping scalars for quantizing first tensors of a first layer 100 that is coupled between two processing layers 100 of the multiple processing layers 100 are computed. In an embodiment, the first tensors are at least one of weights or activations. In an embodiment, the first clipping scalars include a separate scalar for each channel of the first tensors. In an embodiment, the first tensors are decomposed into sub-tensors and the first clipping scalars include a separate scalar for each sub-tensor of the first tensors. In an embodiment, the first tensors are decomposed into vectors and the first clipping scalars include a separate scalar for each vector of the first tensors. In an embodiment, the first clipping scalars are initialized to random values.

In an embodiment, the first clipping scalars are computed according to a Newton-Raphson algorithm. In an embodiment, the first clipping scalars are computed by minimizing a quantization mean squared error. In an embodiment, the first tensors are quantized from a floating-point format to an integer format. In an embodiment, the first tensors are quantized from a floating-point format to a lower precision floating-point format.

At step 160, an input is processed by the neural network model, according to quantized tensors that include the quantized first tensors, by each processing layer of the multiple processing layers in sequence to produce intermediate tensors and an output of the neural network model.

At step 165, the first tensors are adjusted according to a mean squared error to reduce differences between the first tensors and the quantized first tensors. In an embodiment, adjusting the first tensors comprises estimating a loss gradient using the output and applying the loss gradient to update the first tensors. In an embodiment, adjusting the first tensors comprises estimating a loss gradient using the output and applying the loss gradient to update the first tensors. In an embodiment, the loss gradient is estimated using a clipping function that is represented by a magnitude attenuation operation, as described in conjunction with FIGS. 3A and 3B.

At step 170, the first clipping scalars are updated based on the adjusted first tensors. In an embodiment, the first clipping scalars and additional clipping scalars for the intermediate tensors are dynamically or statically updated during training. In an embodiment, updating the first clipping scalars minimized a mean squared error of the differences. In an embodiment, the first clipping scalars and additional

clipping scalars for the intermediate tensors are dynamically or statically updated during inference.

In an embodiment, steps 160, 165, and 170 are repeated for additional inputs. In an embodiment, the intermediate tensors are adjusted according to the mean squared error to reduce differences between the intermediate tensors and the quantized intermediate tensors and second clipping scalars of a second processing layer of the multiple processing layers are updated based on the adjusted intermediate tensors. In an embodiment, at least one of steps of 155, 160, 165, and 170 is performed on a server or in a data center and the output is streamed to a user device. In an embodiment, at least one of steps of 155, 160, 165, and 170 is performed within a cloud computing environment. In an embodiment, at least one of steps of 155, 160, 165, and 170 is performed for training, testing, or certifying the neural network employed in a machine, robot, or autonomous vehicle. In an embodiment, at least one of steps of 155, 160, 165, and 170 is performed on a virtual machine comprising a portion of a graphics processing unit (GPU).

The algorithmic and computational efficiencies of OCTAV make it significantly faster than a conventional brute force search for s^* . On a central processing unit (CPU), and with no code optimizations, OCTAV is measured to be $\sim 10\times$ faster than brute force when applied to weight and activation tensors of a BERT-Base model. Importantly, all operations required by the algorithm are tensor operations. Thus, OCTAV can be implemented on GPUs using any deep learning package. Consequently, OCTAV may be embedded into any QAT routine to realize dynamic quantization using optimal clipping scalars for each tensor at each iteration. The added optimization does incur an overhead, but because OCTAV is fast, it possible to perform the desired QAT in reasonable amounts of time. Additionally, all OCTAV operations are broadcastable and can be used when sub-tensor scaling is required (e.g., for per-output-channel or per-output-feature scaling). Benefitting from the broadcasts, the optimization for finer-grained scaling may incur no slowdown.

FIG. 2A illustrates clipping scalars determined by brute force search compared with the computed optimally clipped scalars, in accordance with an embodiment. A graph 200 illustrates the clipping scalars and computed optimally clipped scalars plotted for tensors of each layer of a pre-trained ResNet-50 model for $B=4,8$. The solid lines in graph 200 indicate the clipping scalars that are determined by a brute force search realized by sweeping the value and s in equation (4). The dashed lines in graph 200 indicate the optimal clipping scalars computed using OCTAV. Consistently, the computed optimally clipped scalars and the brute force determined clipping scalars are either equal or close to one another. Even in the case of a slight mismatch, the resulting quantizers have nearly identical MSE, as illustrated in FIG. 2B.

FIG. 2B illustrates clipping quantization MSE determined by brute force search compared with the computed optimally clipped scalars, in accordance with an embodiment. Once the clipping scalars are determined for FIG. 2A, the clipped quantization MSE is measured empirically using equation (1) to produce the graph 220. The graph 220 plots the empirically measured clipped quantization MSE for the tensors of each layer of the pretrained ResNet-50 neural network model. The solid lines in graph 220 indicate the clipping quantization MSE determined by a brute force search. The dashed lines in graph 220 indicate the clipping quantization MSE for the optimal clipping scalars computed using OCTAV.

Quantizing tensors and vectors processed within a neural network reduces power consumption and may accelerate processing. Derived from the fast Newton-Raphson method, OCTAV provides a recursive algorithm to determine MSE-optimal clipping scalars on the fly, for every tensor, at every iteration of the QAT routine. Thus, the QAT algorithm is formulated with provably minimum quantization noise at each step. QAT using MSE-optimal clipping scalars enables online computation of optimal clipping scalars with fast convergence that provide the same accuracy as the clipping scalars computed offline. Furthermore, the optimal clipping scalars may be computed during inference and/or training using arbitrary initialization. In addition, accuracy may be further improved by compensating for limitations in common gradient estimation techniques in QAT by employing magnitude-aware differentiation.

QAT Gradient Estimation

The OCTAV algorithm enables QAT with minimal noise at each iteration, thereby boosting accuracy. During training of the neural network, the weights are updated based on a loss function via backpropagation of a loss gradient. A gradient estimator is required because the quantization of the tensors is discontinuous and therefore, the gradients may be computed as infinity. The gradient for a layer is defined as the rate of marginal change in loss function for a marginal change in activation (or weight) value. The estimation choice impacts convergence, warranting an analysis of available options.

FIG. 3A illustrates gradient estimation functions for a clipped quantization operation, in accordance with an embodiment. Mathematical limitations of the commonly employed straight-through estimator (STE) and piece-wise linear (PWL) gradients are described for clipped quantization. The limitations may be overcome by using a magnitude-aware derivative (MAD). A graph 300 plots the three gradient estimators: STE, PWL, and MAD, for a clipping scalar $s=1$.

An analysis of gradient back-propagation uses the STE and overlooks equation (3) to

$$\text{set} \frac{\partial^{(STE)} Q(x)}{x} = 1.$$

With clipping, the approximation results in gradient explosion, which causes instability. For an arbitrary activation x_i at layer l ,

$$\Delta x_i = \frac{\partial L}{\partial x_i}$$

defines the true gradient with respect to the loss function $\mathcal{L}$. This gradient is fundamentally defined as the rate of marginal change in loss function for a marginal change in activation value. Further, let $\Delta^{(STE)} x_i$ be the estimate of Δx_i under STE. The following result holds. Namely, in an L -layer network, there exists a positive δ such that the ratio of variances of STE gradient $\text{Var}(\Delta^{(STE)} X_i)$ to true gradient $\text{Var}(\Delta X_i)$ at layer l is lower bounded by:

$$\frac{\text{Var}(\Delta^{(STE)} X_i)}{\text{Var}(\Delta X_i)} \geq 1 + \delta^{L-1} \quad \text{Eq. (7)}$$

The main insight is that STE carries excess variance due to its assigning unity to gradients of clipped weight, as shown by line 305 in the graph 300.

The result in equation (7) highlights an exponential explosion of back-propagated STE gradients. In contrast, the PWL estimator sets

$$\frac{\partial^{(PWL)} Q(x)}{x} = \mathbb{1}_{x \in [-s, s]}$$

and does not suffer from such gradient explosion. However, weight tensors trained using PWL encounter a partial stoppage of convergence, as early as the first training iteration. Early stopping is equivalent to neural network model size reduction, which can impede the achievable accuracy. The following result holds. Namely, given a statically clipped N -element weight tensor $\vec{w}$, whose gradient is estimated using PWL, only $\vec{N}_w^{(i)}$ of its parameters are leaned at iteration i , and the following inequalities hold.

$$N_w^{(i)} > \vec{N}_w^{(i)} \geq \vec{N}_w^{(i+1)} \quad \text{Eq. (8)}$$

The main insight is that PWL repeatedly zeroes out gradients of clipped weights, halting their updates, as shown by line 310 in the graph 300.

The monotonic decrease in equation (8) requires static quantization. Nevertheless, dynamic quantization exhibits a similar, albeit milder, convergence stoppage, where the first strict inequality in equation (8) also holds.

To formulate an improved gradient estimator, rather than treating clipping as a piece-wise selection, magnitude attenuation may be employed. The clipping operator is given by:

$$\text{clip}(x, -s, s) = \alpha \cdot x \quad \text{Eq. (9)}$$

where

$$\alpha = \mathbb{1}_{\{|x| \leq s\}} + \frac{s}{|x|} \mathbb{1}_{\{|x| > s\}}.$$

The result can readily be obtained by replacing the indicator function by its definition, i.e.:

$$\mathbb{1}_{\{|x| \leq s\}} = (1 - \mathbb{1}_{\{|x| > s\}}) = \begin{cases} 1 & \text{if } |x| \leq s \\ 0 & \text{if } |x| > s \end{cases}$$

Using the above definition for the indicator function, the magnitude-aware derivative (MAD) may be formulated. Treating α as a constant in equation (9), the following is obtained:

$$\frac{\partial^{(MAD)} Q(x)}{x} = \mathbb{1}_{\{|x| \leq s\}} + \frac{s}{|x|} \mathbb{1}_{\{|x| > s\}} \quad \text{Eq. (10)}$$

The line for MAD is shown as line 320 in the graph 300. Lines 310 and 315 correspond to STE and PWL, respectively. The three different gradient estimators (STE, PWL, and MAD) are identical in the discretization region. However, while PWL zeroes out the clipping region, MAD uses a magnitude-aware attenuation factor and is continuous.

Therefore, for a MAD-trained weight tensor $\vec{w}$, $\vec{N}_w^{(i)} = N_w^{(i)}$ is guaranteed at any iteration i , and there is no early stoppage of convergence.

In some measure, PWL and MAD are similar. The former approximates

$$\frac{\partial \mathbb{I}_{\{|x| \leq s\}}}{\partial s} = \frac{\partial \mathbb{I}_{\{|x| > s\}}}{\partial s} = 0.$$

Such a style of approximation is predominant and useful in deep learning, e.g., it is used to train networks with ReLU-like activation functions. Similarly, MAD approximates a combination of indicator functions as being a constant to obtain a useful gradient estimator that evades the limitations of PWL.

As described above, MAD improves differentiation for quantized weights. When MAD may also be used for activations. A difference between the MAD and PWL is the occasional zeroing out of activation gradients under PWL which is desirable for potential regularization. In an embodiment, MAD is used to for weight gradient estimation and PWL is used for activation gradient estimation. In an embodiment, a ResNet-50 model is trained-from-scratch and then retraining using QAT at 4-bits. Accuracy is measured for a full precision baseline, max-scaled QAT, and various OCTAV-enabled QAT gradient estimation schemes. The various OCTAV-enabled QAT gradient estimation schemes include STE, PWL, MAD, and MPH (MAD-PWL hybrid using MAD weights and PWL activations). As shown in TABLE 1 below, OCTAV-enabled QAT improves on max-scaling by ~2.5% and achieves a less-than 1% accuracy drop compared to the full precision baseline. Importantly, the improvement is achieved without modifying the training recipe and/or by adding learned parameters or performing hyperparameter tuning.

TABLE 1

4-bit ResNet-50 accuracy					
Full Precision	Max- Scaling	OCTAV			
		STE	PWL	MAD	MPH
76.07	72.67	67.75	74.31	74.81	75.15

FIG. 3B illustrates a block diagram of an example neural network system 325 suitable for use in implementing some embodiments of the present disclosure. As shown in FIG. 3B, the neural network system 325 includes the neural network 120 of FIG. 1A with a loss function unit 340, and a gradient estimation unit 330 paired with a weight update unit 335 and associated with each layer 100. During supervised training the loss function unit 340 computes a loss gradient based on a reference output (e.g., ground truth) and the output computed by the neural network 120. The loss gradients at each layer 100 may be estimated as a MAD. The gradient estimation unit 330 performs the magnitude-aware magnitude attenuation operation (equation 10) based on the loss gradient and the optimal clipping scalar. The estimated gradients are used by the weight update unit 335 to update the weights at each layer 100. The intermediate activations are also differentiated and used to compute the gradients at each layer 100. In an embodiment, activation gradients are estimated as piece-wise linear (PWL) gradients.

Data clipping is crucial in reducing noise in quantization operations and improving the achievable accuracy of quantization-aware training (QAT). As described, analytically and empirically, OCTAV-enabled QAT improves accuracy of

low-precision training without requiring modifications to the learning algorithm, except for the insertion of quantization operations where appropriate. In contrast, conventional techniques rely on heuristics to set clipping threshold scalars and cannot be shown to be optimal. OCTAV-enabled QAT consistently preserves accuracy at low precision (4-to-6-bits). In an embodiment, OCTAV is combined with quantization-dedicated training recipes, such as distillation, to increase accuracy even further.

The techniques employed by OCTAV may be applied to complexity reduction beyond quantization. OCTAV results from formulating quantization noise as an objective function to be minimized on the fly using the Newton-Raphson algorithm. Similarly, other hardware-aware models, such as those for sparsification, can be rapidly optimized for reduced complexity.

Parallel Processing Architecture

FIG. 4 illustrates a parallel processing unit (PPU) 400, in accordance with an embodiment. The PPU 400 may be used to implement the neural network 120. The PPU 400 may be used to implement one or more of the layers 100, gradient estimation unit 330, weight update unit 335, and loss function unit 340 within the neural network system 325. In an embodiment, a processor such as the PPU 400 may be configured to implement a neural network model. The neural network model may be implemented as software instructions executed by the processor or, in other embodiments, the processor can include a matrix of hardware elements configured to process a set of inputs (e.g., electrical signals representing values) to generate a set of outputs, which can represent activations of the neural network model. In yet other embodiments, the neural network model can be implemented as a combination of software instructions and processing performed by a matrix of hardware elements. Implementing the neural network model can include determining a set of parameters for the neural network model through, e.g., supervised or unsupervised training of the neural network model as well as, or in the alternative, performing inference using the set of parameters to process novel sets of inputs.

In an embodiment, the PPU 400 is a multi-threaded processor that is implemented on one or more integrated circuit devices. The PPU 400 is a latency hiding architecture designed to process many threads in parallel. A thread (e.g., a thread of execution) is an instantiation of a set of instructions configured to be executed by the PPU 400. In an embodiment, the PPU 400 is a graphics processing unit (GPU) configured to implement a graphics rendering pipeline for processing three-dimensional (3D) graphics data in order to generate two-dimensional (2D) image data for display on a display device. In other embodiments, the PPU 400 may be utilized for performing general-purpose computations. While one exemplary parallel processor is provided herein for illustrative purposes, it should be strongly noted that such processor is set forth for illustrative purposes only, and that any processor may be employed to supplement and/or substitute for the same.

One or more PPUs 400 may be configured to accelerate thousands of High Performance Computing (HPC), data center, cloud computing, and machine learning applications. The PPU 400 may be configured to accelerate numerous deep learning systems and applications for autonomous vehicles, simulation, computational graphics such as ray or path tracing, deep learning, high-accuracy speech, image, and text recognition systems, intelligent video analytics,

molecular simulations, drug discovery, disease diagnosis, weather forecasting, big data analytics, astronomy, molecular dynamics simulation, financial modeling, robotics, factory automation, real-time language translation, online search optimizations, and personalized user recommendations, and the like.

As shown in FIG. 4, the PPU 400 includes an Input/Output (I/O) unit 405, a front end unit 415, a scheduler unit 420, a work distribution unit 425, a hub 430, a crossbar (Xbar) 470, one or more general processing clusters (GPCs) 450, and one or more memory partition units 480. The PPU 400 may be connected to a host processor or other PPUs 400 via one or more high-speed NVLink 410 interconnect. The PPU 400 may be connected to a host processor or other peripheral devices via an interconnect 402. The PPU 400 may also be connected to a local memory 404 comprising a number of memory devices. In an embodiment, the local memory may comprise a number of dynamic random access memory (DRAM) devices. The DRAM devices may be configured as a high-bandwidth memory (HBM) subsystem, with multiple DRAM dies stacked within each device.

The NVLink 410 interconnect enables systems to scale and include one or more PPUs 400 combined with one or more CPUs, supports cache coherence between the PPUs 400 and CPUs, and CPU mastering. Data and/or commands may be transmitted by the NVLink 410 through the hub 430 to/from other units of the PPU 400 such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). The NVLink 410 is described in more detail in conjunction with FIG. 5B.

The I/O unit 405 is configured to transmit and receive communications (e.g., commands, data, etc.) from a host processor (not shown) over the interconnect 402. The I/O unit 405 may communicate with the host processor directly via the interconnect 402 or through one or more intermediate devices such as a memory bridge. In an embodiment, the I/O unit 405 may communicate with one or more other processors, such as one or more the PPUs 400 via the interconnect 402. In an embodiment, the I/O unit 405 implements a Peripheral Component Interconnect Express (PCIe) interface for communications over a PCIe bus and the interconnect 402 is a PCIe bus. In alternative embodiments, the I/O unit 405 may implement other types of well-known interfaces for communicating with external devices.

The I/O unit 405 decodes packets received via the interconnect 402. In an embodiment, the packets represent commands configured to cause the PPU 400 to perform various operations. The I/O unit 405 transmits the decoded commands to various other units of the PPU 400 as the commands may specify. For example, some commands may be transmitted to the front end unit 415. Other commands may be transmitted to the hub 430 or other units of the PPU 400 such as one or more copy engines, a video encoder, a video decoder, a power management unit, etc. (not explicitly shown). In other words, the I/O unit 405 is configured to route communications between and among the various logical units of the PPU 400.

In an embodiment, a program executed by the host processor encodes a command stream in a buffer that provides workloads to the PPU 400 for processing. A workload may comprise several instructions and data to be processed by those instructions. The buffer is a region in a memory that is accessible (e.g., read/write) by both the host processor and the PPU 400. For example, the I/O unit 405 may be configured to access the buffer in a system memory connected to the interconnect 402 via memory requests transmitted over the interconnect 402. In an embodiment, the host

processor writes the command stream to the buffer and then transmits a pointer to the start of the command stream to the PPU 400. The front end unit 415 receives pointers to one or more command streams. The front end unit 415 manages the one or more streams, reading commands from the streams and forwarding commands to the various units of the PPU 400.

The front end unit 415 is coupled to a scheduler unit 420 that configures the various GPCs 450 to process tasks defined by the one or more streams. The scheduler unit 420 is configured to track state information related to the various tasks managed by the scheduler unit 420. The state may indicate which GPC 450 a task is assigned to, whether the task is active or inactive, a priority level associated with the task, and so forth. The scheduler unit 420 manages the execution of a plurality of tasks on the one or more GPCs 450.

The scheduler unit 420 is coupled to a work distribution unit 425 that is configured to dispatch tasks for execution on the GPCs 450. The work distribution unit 425 may track a number of scheduled tasks received from the scheduler unit 420. In an embodiment, the work distribution unit 425 manages a pending task pool and an active task pool for each of the GPCs 450. As a GPC 450 finishes the execution of a task, that task is evicted from the active task pool for the GPC 450 and one of the other tasks from the pending task pool is selected and scheduled for execution on the GPC 450. If an active task has been idle on the GPC 450, such as while waiting for a data dependency to be resolved, then the active task may be evicted from the GPC 450 and returned to the pending task pool while another task in the pending task pool is selected and scheduled for execution on the GPC 450.

In an embodiment, a host processor executes a driver kernel that implements an application programming interface (API) that enables one or more applications executing on the host processor to schedule operations for execution on the PPU 400. In an embodiment, multiple compute applications are simultaneously executed by the PPU 400 and the PPU 400 provides isolation, quality of service (QoS), and independent address spaces for the multiple compute applications. An application may generate instructions (e.g., API calls) that cause the driver kernel to generate one or more tasks for execution by the PPU 400. The driver kernel outputs tasks to one or more streams being processed by the PPU 400. Each task may comprise one or more groups of related threads, referred to herein as a warp. In an embodiment, a warp comprises 32 related threads that may be executed in parallel. Cooperating threads may refer to a plurality of threads including instructions to perform the task and that may exchange data through shared memory. The tasks may be allocated to one or more processing units within a GPC 450 and instructions are scheduled for execution by at least one warp.

The work distribution unit 425 communicates with the one or more GPCs 450 via XBar 470. The XBar 470 is an interconnect network that couples many of the units of the PPU 400 to other units of the PPU 400. For example, the XBar 470 may be configured to couple the work distribution unit 425 to a particular GPC 450. Although not shown explicitly, one or more other units of the PPU 400 may also be connected to the XBar 470 via the hub 430.

The tasks are managed by the scheduler unit 420 and dispatched to a GPC 450 by the work distribution unit 425. The GPC 450 is configured to process the task and generate results. The results may be consumed by other tasks within the GPC 450, routed to a different GPC 450 via the XBar

470, or stored in the memory 404. The results can be written to the memory 404 via the memory partition units 480, which implement a memory interface for reading and writing data to/from the memory 404. The results can be transmitted to another PPU 400 or CPU via the NVLink 410. In an embodiment, the PPU 400 includes a number U of memory partition units 480 that is equal to the number of separate and distinct memory devices of the memory 404 coupled to the PPU 400. Each GPC 450 may include a memory management unit to provide translation of virtual addresses into physical addresses, memory protection, and arbitration of memory requests. In an embodiment, the memory management unit provides one or more translation lookaside buffers (TLBs) for performing translation of virtual addresses into physical addresses in the memory 404.

In an embodiment, the memory partition unit 480 includes a Raster Operations (ROP) unit, a level two (L2) cache, and a memory interface that is coupled to the memory 404. The memory interface may implement 32, 64, 128, 1024-bit data buses, or the like, for high-speed data transfer. The PPU 400 may be connected to up to Y memory devices, such as high bandwidth memory stacks or graphics double-data-rate, version 5, synchronous dynamic random access memory, or other types of persistent storage. In an embodiment, the memory interface implements an HBM2 memory interface and Y equals half U. In an embodiment, the HBM2 memory stacks are located on the same physical package as the PPU 400, providing substantial power and area savings compared with conventional GDDR5 SDRAM systems. In an embodiment, each HBM2 stack includes four memory dies and Y equals 4, with each HBM2 stack including two 128-bit channels per die for a total of 8 channels and a data bus width of 1024 bits.

In an embodiment, the memory 404 supports Single-Error Correcting Double-Error Detecting (SECDED) Error Correction Code (ECC) to protect data. ECC provides higher reliability for compute applications that are sensitive to data corruption. Reliability is especially important in large-scale cluster computing environments where PPUs 400 process very large datasets and/or run applications for extended periods.

In an embodiment, the PPU 400 implements a multi-level memory hierarchy. In an embodiment, the memory partition unit 480 supports a unified memory to provide a single unified virtual address space for CPU and PPU 400 memory, enabling data sharing between virtual memory systems. In an embodiment the frequency of accesses by a PPU 400 to memory located on other processors is traced to ensure that memory pages are moved to the physical memory of the PPU 400 that is accessing the pages more frequently. In an embodiment, the NVLink 410 supports address translation services allowing the PPU 400 to directly access a CPU's page tables and providing full access to CPU memory by the PPU 400.

In an embodiment, copy engines transfer data between multiple PPUs 400 or between PPUs 400 and CPUs. The copy engines can generate page faults for addresses that are not mapped into the page tables. The memory partition unit 480 can then service the page faults, mapping the addresses into the page table, after which the copy engine can perform the transfer. In a conventional system, memory is pinned (e.g., non-pageable) for multiple copy engine operations between multiple processors, substantially reducing the available memory. With hardware page faulting, addresses can be passed to the copy engines without worrying if the memory pages are resident, and the copy process is transparent.

Data from the memory 404 or other system memory may be fetched by the memory partition unit 480 and stored in the L2 cache 460, which is located on-chip and is shared between the various GPCs 450. As shown, each memory partition unit 480 includes a portion of the L2 cache associated with a corresponding memory 404. Lower level caches may then be implemented in various units within the GPCs 450. For example, each of the processing units within a GPC 450 may implement a level one (L1) cache. The L1 cache is private memory that is dedicated to a particular processing unit. The L2 cache 460 is coupled to the memory interface 470 and the XBar 470 and data from the L2 cache may be fetched and stored in each of the L1 caches for processing.

In an embodiment, the processing units within each GPC 450 implement a SIMD (Single-Instruction, Multiple-Data) architecture where each thread in a group of threads (e.g., a warp) is configured to process a different set of data based on the same set of instructions. All threads in the group of threads execute the same instructions. In another embodiment, the processing unit implements a SIMT (Single-Instruction, Multiple Thread) architecture where each thread in a group of threads is configured to process a different set of data based on the same set of instructions, but where individual threads in the group of threads are allowed to diverge during execution. In an embodiment, a program counter, call stack, and execution state is maintained for each warp, enabling concurrency between warps and serial execution within warps when threads within the warp diverge. In another embodiment, a program counter, call stack, and execution state is maintained for each individual thread, enabling equal concurrency between all threads, within and between warps. When execution state is maintained for each individual thread, threads executing the same instructions may be converged and executed in parallel for maximum efficiency.

Cooperative Groups is a programming model for organizing groups of communicating threads that allows developers to express the granularity at which threads are communicating, enabling the expression of richer, more efficient parallel decompositions. Cooperative launch APIs support synchronization amongst thread blocks for the execution of parallel algorithms. Conventional programming models provide a single, simple construct for synchronizing cooperating threads: a barrier across all threads of a thread block (e.g., the `syncthreads()` function). However, programmers would often like to define groups of threads at smaller than thread block granularities and synchronize within the defined groups to enable greater performance, design flexibility, and software reuse in the form of collective group-wide function interfaces.

Cooperative Groups enables programmers to define groups of threads explicitly at sub-block (e.g., as small as a single thread) and multi-block granularities, and to perform collective operations such as synchronization on the threads in a cooperative group. The programming model supports clean composition across software boundaries, so that libraries and utility functions can synchronize safely within their local context without having to make assumptions about convergence. Cooperative Groups primitives enable new patterns of cooperative parallelism, including producer-consumer parallelism, opportunistic parallelism, and global synchronization across an entire grid of thread blocks.

Each processing unit includes a large number (e.g., 128, etc.) of distinct processing cores (e.g., functional units) that may be fully-pipelined, single-precision, double-precision, and/or mixed precision and include a floating point arith-

metic logic unit and an integer arithmetic logic unit. In an embodiment, the floating point arithmetic logic units implement the IEEE 754-2008 standard for floating point arithmetic. In an embodiment, the cores include 64 single-precision (32-bit) floating point cores, 64 integer cores, 32 double-precision (64-bit) floating point cores, and 8 tensor cores.

Tensor cores configured to perform matrix operations. In particular, the tensor cores are configured to perform deep learning matrix arithmetic, such as GEMM (matrix-matrix multiplication) for convolution operations during neural network training and inferencing. In an embodiment, each tensor core operates on a 4×4 matrix and performs a matrix multiply and accumulate operation $D=A \times B + C$, where A, B, C, and D are 4×4 matrices.

In an embodiment, the matrix multiply inputs A and B may be integer, fixed-point, or floating point matrices, while the accumulation matrices C and D may be integer, fixed-point, or floating point matrices of equal or higher bitwidths. In an embodiment, tensor cores operate on one, four, or eight bit integer input data with 32-bit integer accumulation. The 8-bit integer matrix multiply requires 1024 operations and results in a full precision product that is then accumulated using 32-bit integer addition with the other intermediate products for a 8×8×16 matrix multiply. In an embodiment, tensor Cores operate on 16-bit floating point input data with 32-bit floating point accumulation. The 16-bit floating point multiply requires 64 operations and results in a full precision product that is then accumulated using 32-bit floating point addition with the other intermediate products for a 4×4×4 matrix multiply. In practice, Tensor Cores are used to perform much larger two-dimensional or higher dimensional matrix operations, built up from these smaller elements. An API, such as CUDA 9 C++ API, exposes specialized matrix load, matrix multiply and accumulate, and matrix store operations to efficiently use Tensor Cores from a CUDA-C++ program. At the CUDA level, the warp-level interface assumes 16×16 size matrices spanning all 32 threads of the warp.

Each processing unit may also comprise M special function units (SFUs) that perform special functions (e.g., attribute evaluation, reciprocal square root, and the like). In an embodiment, the SFUs may include a tree traversal unit configured to traverse a hierarchical tree data structure. In an embodiment, the SFUs may include texture unit configured to perform texture map filtering operations. In an embodiment, the texture units are configured to load texture maps (e.g., a 2D array of texels) from the memory 404 and sample the texture maps to produce sampled texture values for use in shader programs executed by the processing unit. In an embodiment, the texture maps are stored in shared memory that may comprise or include an L1 cache. The texture units implement texture operations such as filtering operations using mip-maps (e.g., texture maps of varying levels of detail). In an embodiment, each processing unit includes two texture units.

Each processing unit also comprises N load store units (LSUs) that implement load and store operations between the shared memory and the register file. Each processing unit includes an interconnect network that connects each of the cores to the register file and the LSU to the register file, shared memory. In an embodiment, the interconnect network is a crossbar that can be configured to connect any of the cores to any of the registers in the register file and connect the LSUs to the register file and memory locations in shared memory.

The shared memory is an array of on-chip memory that allows for data storage and communication between the processing units and between threads within a processing unit. In an embodiment, the shared memory comprises 128 KB of storage capacity and is in the path from each of the processing units to the memory partition unit 480. The shared memory can be used to cache reads and writes. One or more of the shared memory, L1 cache, L2 cache, and memory 404 are backing stores.

Combining data cache and shared memory functionality into a single memory block provides the best overall performance for both types of memory accesses. The capacity is usable as a cache by programs that do not use shared memory. For example, if shared memory is configured to use half of the capacity, texture and load/store operations can use the remaining capacity. Integration within the shared memory enables the shared memory to function as a high-throughput conduit for streaming data while simultaneously providing high-bandwidth and low-latency access to frequently reused data.

When configured for general purpose parallel computation, a simpler configuration can be used compared with graphics processing. Specifically, fixed function graphics processing units, are bypassed, creating a much simpler programming model. In the general purpose parallel computation configuration, the work distribution unit 425 assigns and distributes blocks of threads directly to the processing units within the GPCs 450. Threads execute the same program, using a unique thread ID in the calculation to ensure each thread generates unique results, using the processing unit(s) to execute the program and perform calculations, shared memory to communicate between threads, and the LSU to read and write global memory through the shared memory and the memory partition unit 480. When configured for general purpose parallel computation, the processing units can also write commands that the scheduler unit 420 can use to launch new work on the processing units.

The PPUs 400 may each include, and/or be configured to perform functions of, one or more processing cores and/or components thereof, such as Tensor Cores (TCs), Tensor Processing Units (TPUs), Pixel Visual Cores (PVCs), Ray Tracing (RT) Cores, Vision Processing Units (VPUs), Graphics Processing Clusters (GPCs), Texture Processing Clusters (TPCs), Streaming Multiprocessors (SMs), Tree Traversal Units (TTUs), Artificial Intelligence Accelerators (AIAs), Deep Learning Accelerators (DLAs), Arithmetic-Logic Units (ALUs), Application-Specific Integrated Circuits (ASICs), Floating Point Units (FPUs), input/output (I/O) elements, peripheral component interconnect (PCI) or peripheral component interconnect express (PCIe) elements, and/or the like.

The PPU 400 may be included in a desktop computer, a laptop computer, a tablet computer, servers, supercomputers, a smart-phone (e.g., a wireless, hand-held device), personal digital assistant (PDA), a digital camera, a vehicle, a head mounted display, a hand-held electronic device, and the like. In an embodiment, the PPU 400 is embodied on a single semiconductor substrate. In another embodiment, the PPU 400 is included in a system-on-a-chip (SoC) along with one or more other devices such as additional PPUs 400, the memory 404, a reduced instruction set computer (RISC) CPU, a memory management unit (MMU), a digital-to-analog converter (DAC), and the like.

In an embodiment, the PPU 400 may be included on a graphics card that includes one or more memory devices. The graphics card may be configured to interface with a PCIe slot on a motherboard of a desktop computer. In yet

another embodiment, the PPU 400 may be an integrated graphics processing unit (iGPU) or parallel processor included in the chipset of the motherboard. In yet another embodiment, the PPU 400 may be realized in reconfigurable hardware. In yet another embodiment, parts of the PPU 400 may be realized in reconfigurable hardware.

Exemplary Computing System

Systems with multiple GPUs and CPUs are used in a variety of industries as developers expose and leverage more parallelism in applications such as artificial intelligence computing. High-performance GPU-accelerated systems with tens to many thousands of compute nodes are deployed in data centers, research facilities, and supercomputers to solve ever larger problems. As the number of processing devices within the high-performance systems increases, the communication and data transfer mechanisms need to scale to support the increased bandwidth.

FIG. 5A is a conceptual diagram of a processing system 500 implemented using the PPU 400 of FIG. 4, in accordance with an embodiment. The exemplary system 500 may be configured to implement the method 150 shown in FIG. 1C. The processing system 500 includes a CPU 530, switch 510, and multiple PPUs 400, and respective memories 404.

The NVLink 410 provides high-speed communication links between each of the PPUs 400. Although a particular number of NVLink 410 and interconnect 402 connections are illustrated in FIG. 5B, the number of connections to each PPU 400 and the CPU 530 may vary. The switch 510 interfaces between the interconnect 402 and the CPU 530. The PPUs 400, memories 404, and NVLinks 410 may be situated on a single semiconductor platform to form a parallel processing module 525. In an embodiment, the switch 510 supports two or more protocols to interface between various different connections and/or links.

In another embodiment (not shown), the NVLink 410 provides one or more high-speed communication links between each of the PPUs 400 and the CPU 530 and the switch 510 interfaces between the interconnect 402 and each of the PPUs 400. The PPUs 400, memories 404, and interconnect 402 may be situated on a single semiconductor platform to form a parallel processing module 525. In yet another embodiment (not shown), the interconnect 402 provides one or more communication links between each of the PPUs 400 and the CPU 530 and the switch 510 interfaces between each of the PPUs 400 using the NVLink 410 to provide one or more high-speed communication links between the PPUs 400. In another embodiment (not shown), the NVLink 410 provides one or more high-speed communication links between the PPUs 400 and the CPU 530 through the switch 510. In yet another embodiment (not shown), the interconnect 402 provides one or more communication links between each of the PPUs 400 directly. One or more of the NVLink 410 high-speed communication links may be implemented as a physical NVLink interconnect or either an on-chip or on-die interconnect using the same protocol as the NVLink 410.

In the context of the present description, a single semiconductor platform may refer to a sole unitary semiconductor-based integrated circuit fabricated on a die or chip. It should be noted that the term single semiconductor platform may also refer to multi-chip modules with increased connectivity which simulate on-chip operation and make substantial improvements over utilizing a conventional bus implementation. Of course, the various circuits or devices may also be situated separately or in various combinations

of semiconductor platforms per the desires of the user. Alternately, the parallel processing module 525 may be implemented as a circuit board substrate and each of the PPUs 400 and/or memories 404 may be packaged devices. In an embodiment, the CPU 530, switch 510, and the parallel processing module 525 are situated on a single semiconductor platform.

In an embodiment, the signaling rate of each NVLink 410 is 20 to 25 Gigabits/second and each PPU 400 includes six NVLink 410 interfaces (as shown in FIG. 5A, five NVLink 410 interfaces are included for each PPU 400). Each NVLink 410 provides a data transfer rate of 25 Gigabytes/second in each direction, with six links providing 400 Gigabytes/second. The NVLinks 410 can be used exclusively for PPU-to-PPU communication as shown in FIG. 5A, or some combination of PPU-to-PPU and PPU-to-CPU, when the CPU 530 also includes one or more NVLink 410 interfaces.

In an embodiment, the NVLink 410 allows direct load/store/atomic access from the CPU 530 to each PPU's 400 memory 404. In an embodiment, the NVLink 410 supports coherency operations, allowing data read from the memories 404 to be stored in the cache hierarchy of the CPU 530, reducing cache access latency for the CPU 530. In an embodiment, the NVLink 410 includes support for Address Translation Services (ATS), allowing the PPU 400 to directly access page tables within the CPU 530. One or more of the NVLinks 410 may also be configured to operate in a low-power mode.

FIG. 5B illustrates an exemplary system 565 in which the various architecture and/or functionality of the various previous embodiments may be implemented. The exemplary system 565 may be configured to implement the method 150 shown in FIG. 1C. As shown, a system 565 is provided including at least one central processing unit 530 that is connected to a communication bus 575. The communication bus 575 may directly or indirectly couple one or more of the following devices: main memory 540, network interface 535, CPU(s) 530, display device(s) 545, input device(s) 560, switch 510, and parallel processing system 525. The communication bus 575 may be implemented using any suitable protocol and may represent one or more links or busses, such as an address bus, a data bus, a control bus, or a combination thereof. The communication bus 575 may include one or more bus or link types, such as an industry standard architecture (ISA) bus, an extended industry standard architecture (EISA) bus, a video electronics standards association (VESA) bus, a peripheral component interconnect (PCI) bus, a peripheral component interconnect express (PCIe) bus, HyperTransport, and/or another type of bus or link. In some embodiments, there are direct connections between components. As an example, the CPU(s) 530 may be directly connected to the main memory 540. Further, the CPU(s) 530 may be directly connected to the parallel processing system 525. Where there is direct, or point-to-point connection between components, the communication bus 575 may include a PCIe link to carry out the connection. In these examples, a PCI bus need not be included in the system 565.

Although the various blocks of FIG. 5B are shown as connected via the communication bus 575 with lines, this is not intended to be limiting and is for clarity only. For example, in some embodiments, a presentation component, such as display device(s) 545, may be considered an I/O component, such as input device(s) 560 (e.g., if the display is a touch screen). As another example, the CPU(s) 530 and/or parallel processing system 525 may include memory (e.g., the main memory 540 may be representative of a

storage device in addition to the parallel processing system 525, the CPUs 530, and/or other components). In other words, the computing device of FIG. 5B is merely illustrative. Distinction is not made between such categories as “workstation,” “server,” “laptop,” “desktop,” “tablet,” “client device,” “mobile device,” “hand-held device,” “game console,” “electronic control unit (ECU),” “virtual reality system,” and/or other device or system types, as all are contemplated within the scope of the computing device of FIG. 5B.

The system 565 also includes a main memory 540. Control logic (software) and data are stored in the main memory 540 which may take the form of a variety of computer-readable media. The computer-readable media may be any available media that may be accessed by the system 565. The computer-readable media may include both volatile and nonvolatile media, and removable and non-removable media. By way of example, and not limitation, the computer-readable media may comprise computer-storage media and communication media.

The computer-storage media may include both volatile and nonvolatile media and/or removable and non-removable media implemented in any method or technology for storage of information such as computer-readable instructions, data structures, program modules, and/or other data types. For example, the main memory 540 may store computer-readable instructions (e.g., that represent a program(s) and/or a program element(s), such as an operating system. Computer-storage media may include, but is not limited to, RAM, ROM, EEPROM, flash memory or other memory technology, CD-ROM, digital versatile disks (DVD) or other optical disk storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices, or any other medium which may be used to store the desired information and which may be accessed by system 565. As used herein, computer storage media does not comprise signals per se.

The computer storage media may embody computer-readable instructions, data structures, program modules, and/or other data types in a modulated data signal such as a carrier wave or other transport mechanism and includes any information delivery media. The term “modulated data signal” may refer to a signal that has one or more of its characteristics set or changed in such a manner as to encode information in the signal. By way of example, and not limitation, the computer storage media may include wired media such as a wired network or direct-wired connection, and wireless media such as acoustic, RF, infrared and other wireless media. Combinations of any of the above should also be included within the scope of computer-readable media.

Computer programs, when executed, enable the system 565 to perform various functions. The CPU(s) 530 may be configured to execute at least some of the computer-readable instructions to control one or more components of the system 565 to perform one or more of the methods and/or processes described herein. The CPU(s) 530 may each include one or more cores (e.g., one, two, four, eight, twenty-eight, seventy-two, etc.) that are capable of handling a multitude of software threads simultaneously. The CPU(s) 530 may include any type of processor, and may include different types of processors depending on the type of system 565 implemented (e.g., processors with fewer cores for mobile devices and processors with more cores for servers). For example, depending on the type of system 565, the processor may be an Advanced RISC Machines (ARM) processor implemented using Reduced Instruction Set Computing (RISC) or an x86 processor implemented using

Complex Instruction Set Computing (CISC). The system 565 may include one or more CPUs 530 in addition to one or more microprocessors or supplementary co-processors, such as math co-processors.

In addition to or alternatively from the CPU(s) 530, the parallel processing module 525 may be configured to execute at least some of the computer-readable instructions to control one or more components of the system 565 to perform one or more of the methods and/or processes described herein. The parallel processing module 525 may be used by the system 565 to render graphics (e.g., 3D graphics) or perform general purpose computations. For example, the parallel processing module 525 may be used for General-Purpose computing on GPUs (GPGPU). In embodiments, the CPU(s) 530 and/or the parallel processing module 525 may discretely or jointly perform any combination of the methods, processes and/or portions thereof.

The system 565 also includes input device(s) 560, the parallel processing system 525, and display device(s) 545. The display device(s) 545 may include a display (e.g., a monitor, a touch screen, a television screen, a heads-up-display (HUD), other display types, or a combination thereof), speakers, and/or other presentation components. The display device(s) 545 may receive data from other components (e.g., the parallel processing system 525, the CPU(s) 530, etc.), and output the data (e.g., as an image, video, sound, etc.).

The network interface 535 may enable the system 565 to be logically coupled to other devices including the input devices 560, the display device(s) 545, and/or other components, some of which may be built in to (e.g., integrated in) the system 565. Illustrative input devices 560 include a microphone, mouse, keyboard, joystick, game pad, game controller, satellite dish, scanner, printer, wireless device, etc. The input devices 560 may provide a natural user interface (NUI) that processes air gestures, voice, or other physiological inputs generated by a user. In some instances, inputs may be transmitted to an appropriate network element for further processing. An NUI may implement any combination of speech recognition, stylus recognition, facial recognition, biometric recognition, gesture recognition both on screen and adjacent to the screen, air gestures, head and eye tracking, and touch recognition (as described in more detail below) associated with a display of the system 565. The system 565 may include depth cameras, such as stereoscopic camera systems, infrared camera systems, RGB camera systems, touchscreen technology, and combinations of these, for gesture detection and recognition. Additionally, the system 565 may include accelerometers or gyroscopes (e.g., as part of an inertia measurement unit (IMU)) that enable detection of motion. In some examples, the output of the accelerometers or gyroscopes may be used by the system 565 to render immersive augmented reality or virtual reality.

Further, the system 565 may be coupled to a network (e.g., a telecommunications network, local area network (LAN), wireless network, wide area network (WAN) such as the Internet, peer-to-peer network, cable network, or the like) through a network interface 535 for communication purposes. The system 565 may be included within a distributed network and/or cloud computing environment.

The network interface 535 may include one or more receivers, transmitters, and/or transceivers that enable the system 565 to communicate with other computing devices via an electronic communication network, included wired and/or wireless communications. The network interface 535 may be implemented as a network interface controller (NIC) that includes one or more data processing units (DPUs) to

perform operations such as (for example and without limitation) packet parsing and accelerating network processing and communication. The network interface 535 may include components and functionality to enable communication over any of a number of different networks, such as wireless networks (e.g., Wi-Fi, Z-Wave, Bluetooth, Bluetooth LE, ZigBee, etc.), wired networks (e.g., communicating over Ethernet or InfiniBand), low-power wide-area networks (e.g., LoRaWAN, SigFox, etc.), and/or the Internet.

The system 565 may also include a secondary storage (not shown). The secondary storage includes, for example, a hard disk drive and/or a removable storage drive, representing a floppy disk drive, a magnetic tape drive, a compact disk drive, digital versatile disk (DVD) drive, recording device, universal serial bus (USB) flash memory. The removable storage drive reads from and/or writes to a removable storage unit in a well-known manner. The system 565 may also include a hard-wired power supply, a battery power supply, or a combination thereof (not shown). The power supply may provide power to the system 565 to enable the components of the system 565 to operate.

Each of the foregoing modules and/or devices may even be situated on a single semiconductor platform to form the system 565. Alternately, the various modules may also be situated separately or in various combinations of semiconductor platforms per the desires of the user. While various embodiments have been described above, it should be understood that they have been presented by way of example only, and not limitation. Thus, the breadth and scope of a preferred embodiment should not be limited by any of the above-described exemplary embodiments, but should be defined only in accordance with the following claims and their equivalents.

Example Network Environments

Network environments suitable for use in implementing embodiments of the disclosure may include one or more client devices, servers, network attached storage (NAS), other backend devices, and/or other device types. The client devices, servers, and/or other device types (e.g., each device) may be implemented on one or more instances of the processing system 500 of FIG. 5A and/or exemplary system 565 of FIG. 5B—e.g., each device may include similar components, features, and/or functionality of the processing system 500 and/or exemplary system 565.

Components of a network environment may communicate with each other via a network(s), which may be wired, wireless, or both. The network may include multiple networks, or a network of networks. By way of example, the network may include one or more Wide Area Networks (WANs), one or more Local Area Networks (LANs), one or more public networks such as the Internet and/or a public switched telephone network (PSTN), and/or one or more private networks. Where the network includes a wireless telecommunications network, components such as a base station, a communications tower, or even access points (as well as other components) may provide wireless connectivity.

Compatible network environments may include one or more peer-to-peer network environments—in which case a server may not be included in a network environment—and one or more client-server network environments—in which case one or more servers may be included in a network environment. In peer-to-peer network environments, functionality described herein with respect to a server(s) may be implemented on any number of client devices.

In at least one embodiment, a network environment may include one or more cloud-based network environments, a distributed computing environment, a combination thereof, etc. A cloud-based network environment may include a framework layer, a job scheduler, a resource manager, and a distributed file system implemented on one or more of servers, which may include one or more core network servers and/or edge servers. A framework layer may include a framework to support software of a software layer and/or one or more application(s) of an application layer. The software or application(s) may respectively include web-based service software or applications. In embodiments, one or more of the client devices may use the web-based service software or applications (e.g., by accessing the service software and/or applications via one or more application programming interfaces (APIs)). The framework layer may be, but is not limited to, a type of free and open-source software web application framework such as that may use a distributed file system for large-scale data processing (e.g., “big data”).

A cloud-based network environment may provide cloud computing and/or cloud storage that carries out any combination of computing and/or data storage functions described herein (or one or more portions thereof). Any of these various functions may be distributed over multiple locations from central or core servers (e.g., of one or more data centers that may be distributed across a state, a region, a country, the globe, etc.). If a connection to a user (e.g., a client device) is relatively close to an edge server(s), a core server(s) may designate at least a portion of the functionality to the edge server(s). A cloud-based network environment may be private (e.g., limited to a single organization), may be public (e.g., available to many organizations), and/or a combination thereof (e.g., a hybrid cloud environment).

The client device(s) may include at least some of the components, features, and functionality of the example processing system 500 of FIG. 5A and/or exemplary system 565 of FIG. 5B. By way of example and not limitation, a client device may be embodied as a Personal Computer (PC), a laptop computer, a mobile device, a smartphone, a tablet computer, a smart watch, a wearable computer, a Personal Digital Assistant (PDA), an MP3 player, a virtual reality headset, a Global Positioning System (GPS) or device, a video player, a video camera, a surveillance device or system, a vehicle, a boat, a flying vessel, a virtual machine, a drone, a robot, a handheld communications device, a hospital device, a gaming device or system, an entertainment system, a vehicle computer system, an embedded system controller, a remote control, an appliance, a consumer electronic device, a workstation, an edge device, any combination of these delineated devices, or any other suitable device.

Machine Learning

Deep neural networks (DNNs) developed on processors, such as the PPU 400 have been used for diverse use cases, from self-driving cars to faster drug development, from automatic image captioning in online image databases to smart real-time language translation in video chat applications. Deep learning is a technique that models the neural learning process of the human brain, continually learning, continually getting smarter, and delivering more accurate results more quickly over time. A child is initially taught by an adult to correctly identify and classify various shapes, eventually being able to identify shapes without any coaching. Similarly, a deep learning or neural learning system

needs to be trained in object recognition and classification for it get smarter and more efficient at identifying basic objects, occluded objects, etc., while also assigning context to objects.

At the simplest level, neurons in the human brain look at various inputs that are received, importance levels are assigned to each of these inputs, and output is passed on to other neurons to act upon. An artificial neuron is the most basic model of a neural network. In one example, a neuron may receive one or more inputs that represent various features of an object that the neuron is being trained to recognize and classify, and each of these features is assigned a certain weight based on the importance of that feature in defining the shape of an object.

A deep neural network (DNN) model includes multiple layers of many connected nodes (e.g., Boltzmann machines, radial basis functions, convolutional layers, etc.) that can be trained with enormous amounts of input data to quickly solve complex problems with high accuracy. In one example, a first layer of the DNN model breaks down an input image of an automobile into various sections and looks for basic patterns such as lines and angles. The second layer assembles the lines to look for higher level patterns such as wheels, windshields, and mirrors. The next layer identifies the type of vehicle, and the final few layers generate a label for the input image, identifying the model of a specific automobile brand.

Once the DNN is trained, the DNN can be deployed and used to identify and classify objects or patterns in a process known as inference. Examples of inference (the process through which a DNN extracts useful information from a given input) include identifying handwritten numbers on checks deposited into ATM machines, identifying images of friends in photos, delivering movie recommendations to over fifty million users, identifying and classifying different types of automobiles, pedestrians, and road hazards in driverless cars, or translating human speech in real-time.

During training, data flows through the DNN in a forward propagation phase until a prediction is produced that indicates a label corresponding to the input. If the neural network does not correctly label the input, then errors between the correct label and the predicted label are analyzed, and the weights are adjusted for each feature during a backward propagation phase until the DNN correctly labels the input and other inputs in a training dataset. Training complex neural networks requires massive amounts of parallel computing performance, including floating-point multiplications and additions that are supported by the PPU 400. Inferencing is less compute-intensive than training, being a latency-sensitive process where a trained neural network is applied to new inputs it has not seen before to classify images, detect emotions, identify recommendations, recognize and translate speech, and generally infer new information.

Neural networks rely heavily on matrix math operations, and complex multi-layered networks require tremendous amounts of floating-point performance and bandwidth for both efficiency and speed. With thousands of processing cores, optimized for matrix math operations, and delivering tens to hundreds of TFLOPS of performance, the PPU 400 is a computing platform capable of delivering performance required for deep neural network-based artificial intelligence and machine learning applications.

Furthermore, images generated applying one or more of the techniques disclosed herein may be used to train, test, or certify DNNs used to recognize objects and environments in the real world. Such images may include scenes of road-

ways, factories, buildings, urban settings, rural settings, humans, animals, and any other physical object or real-world setting. Such images may be used to train, test, or certify DNNs that are employed in machines or robots to manipulate, handle, or modify physical objects in the real world. Furthermore, such images may be used to train, test, or certify DNNs that are employed in autonomous vehicles to navigate and move the vehicles through the real world. Additionally, images generated applying one or more of the techniques disclosed herein may be used to convey information to users of such machines, robots, and vehicles.

FIG. 5C illustrates components of an exemplary system 555 that can be used to train and utilize machine learning, in accordance with at least one embodiment. As will be discussed, various components can be provided by various combinations of computing devices and resources, or a single computing system, which may be under control of a single entity or multiple entities. Further, aspects may be triggered, initiated, or requested by different entities. In at least one embodiment training of a neural network might be instructed by a provider associated with provider environment 506, while in at least one embodiment training might be requested by a customer or other user having access to a provider environment through a client device 502 or other such resource. In at least one embodiment, training data (or data to be analyzed by a trained neural network) can be provided by a provider, a user, or a third party content provider 524. In at least one embodiment, client device 502 may be a vehicle or object that is to be navigated on behalf of a user, for example, which can submit requests and/or receive instructions that assist in navigation of a device.

In at least one embodiment, requests are able to be submitted across at least one network 504 to be received by a provider environment 506. In at least one embodiment, a client device may be any appropriate electronic and/or computing devices enabling a user to generate and send such requests, such as, but not limited to, desktop computers, notebook computers, computer servers, smartphones, tablet computers, gaming consoles (portable or otherwise), computer processors, computing logic, and set-top boxes. Network(s) 504 can include any appropriate network for transmitting a request or other such data, as may include Internet, an intranet, an Ethernet, a cellular network, a local area network (LAN), a wide area network (WAN), a personal area network (PAN), an ad hoc network of direct wireless connections among peers, and so on.

In at least one embodiment, requests can be received at an interface layer 508, which can forward data to a training and inference manager 532, in this example. The training and inference manager 532 can be a system or service including hardware and software for managing requests and service corresponding data or content, in at least one embodiment, the training and inference manager 532 can receive a request to train a neural network, and can provide data for a request to a training module 512. In at least one embodiment, training module 512 can select an appropriate model or neural network to be used, if not specified by the request, and can train a model using relevant training data. In at least one embodiment, training data can be a batch of data stored in a training data repository 514, received from client device 502, or obtained from a third party provider 524. In at least one embodiment, training module 512 can be responsible for training data. A neural network can be any appropriate network, such as a recurrent neural network (RNN) or convolutional neural network (CNN). Once a neural network is trained and successfully evaluated, a trained neural network can be stored in a model repository 516, for

example, that may store different models or networks for users, applications, or services, etc. In at least one embodiment, there may be multiple models for a single application or entity, as may be utilized based on a number of different factors.

In at least one embodiment, at a subsequent point in time, a request may be received from client device 502 (or another such device) for content (e.g., path determinations) or data that is at least partially determined or impacted by a trained neural network. This request can include, for example, input data to be processed using a neural network to obtain one or more inferences or other output values, classifications, or predictions, or for at least one embodiment, input data can be received by interface layer 508 and directed to inference module 518, although a different system or service can be used as well. In at least one embodiment, inference module 518 can obtain an appropriate trained network, such as a trained deep neural network (DNN) as discussed herein, from model repository 516 if not already stored locally to inference module 518. Inference module 518 can provide data as input to a trained network, which can then generate one or more inferences as output. This may include, for example, a classification of an instance of input data. In at least one embodiment, inferences can then be transmitted to client device 502 for display or other communication to a user. In at least one embodiment, context data for a user may also be stored to a user context data repository 522, which may include data about a user which may be useful as input to a network in generating inferences, or determining data to return to a user after obtaining instances. In at least one embodiment, relevant data, which may include at least some of input or inference data, may also be stored to a local database 534 for processing future requests. In at least one embodiment, a user can use account information or other information to access resources or functionality of a provider environment. In at least one embodiment, if permitted and available, user data may also be collected and used to further train models, in order to provide more accurate inferences for future requests. In at least one embodiment, requests may be received through a user interface to a machine learning application 526 executing on client device 502, and results displayed through a same interface. A client device can include resources such as a processor 528 and memory 562 for generating a request and processing results or a response, as well as at least one data storage element 552 for storing data for machine learning application 526.

In at least one embodiment a processor 528 (or a processor of training module 512 or inference module 518) will be a central processing unit (CPU). As mentioned, however, resources in such environments can utilize GPUs to process data for at least certain types of requests. With thousands of cores, GPUs, such as PPU 400 are designed to handle substantial parallel workloads and, therefore, have become popular in deep learning for training neural networks and generating predictions. While use of GPUs for offline builds has enabled faster training of larger and more complex models, generating predictions offline implies that either request-time input features cannot be used or predictions must be generated for all permutations of features and stored in a lookup table to serve real-time requests. If a deep learning framework supports a CPU-mode and a model is small and simple enough to perform a feed-forward on a CPU with a reasonable latency, then a service on a CPU instance could host a model. In this case, training can be done offline on a GPU and inference done in real-time on a CPU. If a CPU approach is not viable, then a service can run on a GPU instance. Because GPUs have different perfor-

mance and cost characteristics than CPUs, however, running a service that offloads a runtime algorithm to a GPU can require it to be designed differently from a CPU based service.

In at least one embodiment, video data can be provided from client device 502 for enhancement in provider environment 506. In at least one embodiment, video data can be processed for enhancement on client device 502. In at least one embodiment, video data may be streamed from a third party content provider 524 and enhanced by third party content provider 524, provider environment 506, or client device 502. In at least one embodiment, video data can be provided from client device 502 for use as training data in provider environment 506.

In at least one embodiment, supervised and/or unsupervised training can be performed by the client device 502 and/or the provider environment 506. In at least one embodiment, a set of training data 514 (e.g., classified or labeled data) is provided as input to function as training data. In at least one embodiment, training data can include instances of at least one type of object for which a neural network is to be trained, as well as information that identifies that type of object. In at least one embodiment, training data might include a set of images that each includes a representation of a type of object, where each image also includes, or is associated with, a label, metadata, classification, or other piece of information identifying a type of object represented in a respective image. Various other types of data may be used as training data as well, as may include text data, audio data, video data, and so on. In at least one embodiment, training data 514 is provided as training input to a training module 512. In at least one embodiment, training module 512 can be a system or service that includes hardware and software, such as one or more computing devices executing a training application, for training a neural network (or other model or algorithm, etc.). In at least one embodiment, training module 512 receives an instruction or request indicating a type of model to be used for training, in at least one embodiment, a model can be any appropriate statistical model, network, or algorithm useful for such purposes, as may include an artificial neural network, deep learning algorithm, learning classifier, Bayesian network, and so on. In at least one embodiment, training module 512 can select an initial model, or other untrained model, from an appropriate repository 516 and utilize training data 514 to train a model, thereby generating a trained model (e.g., trained deep neural network) that can be used to classify similar types of data, or generate other such inferences. In at least one embodiment where training data is not used, an appropriate initial model can still be selected for training on input data per training module 512.

In at least one embodiment, a model can be trained in a number of different ways, as may depend in part upon a type of model selected. In at least one embodiment, a machine learning algorithm can be provided with a set of training data, where a model is a model artifact created by a training process. In at least one embodiment, each instance of training data contains a correct answer (e.g., classification), which can be referred to as a target or target attribute. In at least one embodiment, a learning algorithm finds patterns in training data that map input data attributes to a target, an answer to be predicted, and a machine learning model is output that captures these patterns. In at least one embodiment, a machine learning model can then be used to obtain predictions on new data for which a target is not specified.

In at least one embodiment, training and inference manager 532 can select from a set of machine learning models

including binary classification, multiclass classification, generative, and regression models. In at least one embodiment, a type of model to be used can depend at least in part upon a type of target to be predicted.

Graphics Processing Pipeline

In an embodiment, the PPU 400 comprises a graphics processing unit (GPU). The PPU 400 is configured to receive commands that specify shader programs for processing graphics data. Graphics data may be defined as a set of primitives such as points, lines, triangles, quads, triangle strips, and the like. Typically, a primitive includes data that specifies a number of vertices for the primitive (e.g., in a model-space coordinate system) as well as attributes associated with each vertex of the primitive. The PPU 400 can be configured to process the graphics primitives to generate a frame buffer (e.g., pixel data for each of the pixels of the display).

An application writes model data for a scene (e.g., a collection of vertices and attributes) to a memory such as a system memory or memory 404. The model data defines each of the objects that may be visible on a display. The application then makes an API call to the driver kernel that requests the model data to be rendered and displayed. The driver kernel reads the model data and writes commands to the one or more streams to perform operations to process the model data. The commands may reference different shader programs to be implemented on the processing units within the PPU 400 including one or more of a vertex shader, hull shader, domain shader, geometry shader, and a pixel shader. For example, one or more of the processing units may be configured to execute a vertex shader program that processes a number of vertices defined by the model data. In an embodiment, the different processing units may be configured to execute different shader programs concurrently. For example, a first subset of processing units may be configured to execute a vertex shader program while a second subset of processing units may be configured to execute a pixel shader program. The first subset of processing units processes vertex data to produce processed vertex data and writes the processed vertex data to the L2 cache 460 and/or the memory 404. After the processed vertex data is rasterized (e.g., transformed from three-dimensional data into two-dimensional data in screen space) to produce fragment data, the second subset of processing units executes a pixel shader to produce processed fragment data, which is then blended with other processed fragment data and written to the frame buffer in memory 404. The vertex shader program and pixel shader program may execute concurrently, processing different data from the same scene in a pipelined fashion until all of the model data for the scene has been rendered to the frame buffer. Then, the contents of the frame buffer are transmitted to a display controller for display on a display device.

Images generated applying one or more of the techniques disclosed herein may be displayed on a monitor or other display device. In some embodiments, the display device may be coupled directly to the system or processor generating or rendering the images. In other embodiments, the display device may be coupled indirectly to the system or processor such as via a network. Examples of such networks include the Internet, mobile telecommunications networks, a WIFI network, as well as any other wired and/or wireless networking system. When the display device is indirectly coupled, the images generated by the system or processor may be streamed over the network to the display device.

Such streaming allows, for example, video games or other applications, which render images, to be executed on a server, a data center, or in a cloud-based computing environment and the rendered images to be transmitted and displayed on one or more user devices (such as a computer, video game console, smartphone, other mobile device, etc.) that are physically separate from the server or data center. Hence, the techniques disclosed herein can be applied to enhance the images that are streamed and to enhance services that stream images such as NVIDIA GeForce Now (GFN), Google Stadia, and the like.

Example Streaming System

FIG. 6 is an example system diagram for a streaming system 605, in accordance with some embodiments of the present disclosure. FIG. 6 includes server(s) 603 (which may include similar components, features, and/or functionality to the example processing system 500 of FIG. 5A and/or exemplary system 565 of FIG. 5B), client device(s) 604 (which may include similar components, features, and/or functionality to the example processing system 500 of FIG. 5A and/or exemplary system 565 of FIG. 5B), and network(s) 606 (which may be similar to the network(s) described herein). In some embodiments of the present disclosure, the system 605 may be implemented.

In an embodiment, the streaming system 605 is a game streaming system and the server(s) 603 are game server(s). In the system 605, for a game session, the client device(s) 604 may only receive input data in response to inputs to the input device(s) 626, transmit the input data to the server(s) 603, receive encoded display data from the server(s) 603, and display the display data on the display 624. As such, the more computationally intense computing and processing is offloaded to the server(s) 603 (e.g., rendering—in particular ray or path tracing—for graphical output of the game session is executed by the GPU(s) 615 of the server(s) 603). In other words, the game session is streamed to the client device(s) 604 from the server(s) 603, thereby reducing the requirements of the client device(s) 604 for graphics processing and rendering.

For example, with respect to an instantiation of a game session, a client device 604 may be displaying a frame of the game session on the display 624 based on receiving the display data from the server(s) 603. The client device 604 may receive an input to one of the input device(s) 626 and generate input data in response. The client device 604 may transmit the input data to the server(s) 603 via the communication interface 621 and over the network(s) 606 (e.g., the Internet), and the server(s) 603 may receive the input data via the communication interface 618. The CPU(s) 608 may receive the input data, process the input data, and transmit data to the GPU(s) 615 that causes the GPU(s) 615 to generate a rendering of the game session. For example, the input data may be representative of a movement of a character of the user in a game, firing a weapon, reloading, passing a ball, turning a vehicle, etc. The rendering component 612 may render the game session (e.g., representative of the result of the input data) and the render capture component 614 may capture the rendering of the game session as display data (e.g., as image data capturing the rendered frame of the game session). The rendering of the game session may include ray or path-traced lighting and/or shadow effects, computed using one or more parallel processing units—such as GPUs, which may further employ the use of one or more dedicated hardware accelerators or processing cores to perform ray or path-tracing techniques—

31

of the server(s) 603. The encoder 616 may then encode the display data to generate encoded display data and the encoded display data may be transmitted to the client device 604 over the network(s) 606 via the communication interface 618. The client device 604 may receive the encoded display data via the communication interface 621 and the decoder 622 may decode the encoded display data to generate the display data. The client device 604 may then display the display data via the display 624.

It is noted that the techniques described herein may be embodied in executable instructions stored in a computer readable medium for use by or in connection with a processor-based instruction execution machine, system, apparatus, or device. It will be appreciated by those skilled in the art that, for some embodiments, various types of computer-readable media can be included for storing data. As used herein, a “computer-readable medium” includes one or more of any suitable media for storing the executable instructions of a computer program such that the instruction execution machine, system, apparatus, or device may read (or fetch) the instructions from the computer-readable medium and execute the instructions for carrying out the described embodiments. Suitable storage formats include one or more of an electronic, magnetic, optical, and electromagnetic format. A non-exhaustive list of conventional exemplary computer-readable medium includes: a portable computer diskette; a random-access memory (RAM); a read-only memory (ROM); an erasable programmable read only memory (EPROM); a flash memory device; and optical storage devices, including a portable compact disc (CD), a portable digital video disc (DVD), and the like.

It should be understood that the arrangement of components illustrated in the attached Figures are for illustrative purposes and that other arrangements are possible. For example, one or more of the elements described herein may be realized, in whole or in part, as an electronic hardware component. Other elements may be implemented in software, hardware, or a combination of software and hardware. Moreover, some or all of these other elements may be combined, some may be omitted altogether, and additional components may be added while still achieving the functionality described herein. Thus, the subject matter described herein may be embodied in many different variations, and all such variations are contemplated to be within the scope of the claims.

To facilitate an understanding of the subject matter described herein, many aspects are described in terms of sequences of actions. It will be recognized by those skilled in the art that the various actions may be performed by specialized circuits or circuitry, by program instructions being executed by one or more processors, or by a combination of both. The description herein of any sequence of actions is not intended to imply that the specific order described for performing that sequence must be followed. All methods described herein may be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context.

The use of the terms “a” and “an” and “the” and similar references in the context of describing the subject matter (particularly in the context of the following claims) are to be construed to cover both the singular and the plural, unless otherwise indicated herein or clearly contradicted by context. The use of the term “at least one” followed by a list of one or more items (for example, “at least one of A and B”) is to be construed to mean one item selected from the listed items (A or B) or any combination of two or more of the listed items (A and B), unless otherwise indicated herein or

32

clearly contradicted by context. Furthermore, the foregoing description is for the purpose of illustration only, and not for the purpose of limitation, as the scope of protection sought is defined by the claims as set forth hereinafter together with any equivalents thereof. The use of any and all examples, or exemplary language (e.g., “such as”) provided herein, is intended merely to better illustrate the subject matter and does not pose a limitation on the scope of the subject matter unless otherwise claimed. The use of the term “based on” and other like phrases indicating a condition for bringing about a result, both in the claims and in the written description, is not intended to foreclose any other conditions that bring about that result. No language in the specification should be construed as indicating any non-claimed element as essential to the practice of the invention as claimed.

What is claimed is:

1. A computer-implemented method for quantizing tensors of a neural network model comprising multiple processing layers, comprising:

computing first clipping scalars for quantizing first tensors of a first processing layer that is coupled between two processing layers of the multiple processing layers; processing an input by the neural network model, according to quantized tensors that include the quantized first tensors, by each processing layer of the multiple processing layers in sequence to produce intermediate tensors and an output of the neural network model; adjusting the first tensors based on a loss gradient; and updating the first clipping scalars based on a mean squared error to reduce differences between the adjusted first tensors and quantized adjusted first tensors.

2. The computer-implemented method of claim 1, wherein the first tensors are at least one of weights or activations.

3. The computer-implemented method of claim 1, wherein the first clipping scalars are recursively computed within the first processing layer according to a Newton-Raphson algorithm.

4. The computer-implemented method of claim 1, wherein the first clipping scalars are computed by minimizing a quantization mean squared error.

5. The computer-implemented method of claim 1, repeating the processing, adjusting, and updating for additional inputs.

6. The computer-implemented method of claim 1, wherein the first clipping scalars and additional clipping scalars for the intermediate tensors are dynamically or statically updated during training.

7. The computer-implemented method of claim 1, wherein the first clipping scalars and additional clipping scalars for the intermediate tensors are dynamically or statically updated during inference.

8. The computer-implemented method of claim 1, wherein the first tensors are quantized from a floating-point format to an integer format.

9. The computer-implemented method of claim 1, wherein the first tensors are quantized from a floating-point format to a lower precision floating-point format.

10. The computer-implemented method of claim 1, wherein updating the first clipping scalars minimizes a mean squared error of the differences.

11. The computer-implemented method of claim 1, wherein adjusting the first tensors comprises applying the loss gradient and the first clipping scalars to update the first tensors.

33

12. The computer-implemented method of claim 1, further comprising estimating the loss gradient using a magnitude attenuation operation.

13. The computer-implemented method of claim 1, wherein the first clipping scalars include a separate scalar for each channel of the first tensors.

14. The computer-implemented method of claim 1, wherein the first tensors are decomposed into sub-tensors and the first clipping scalars include a separate scalar for each sub-tensor of the first tensors.

15. The computer-implemented method of claim 14, wherein the first tensors are decomposed into vectors and the first clipping scalars include a separate scalar for each vector of the first tensors.

16. The computer-implemented method of claim 1, further comprising:

adjusting the intermediate tensors based on the loss gradient; and

updating second clipping scalars of a second processing layer of the multiple processing layers based on the mean squared error to reduce differences between the adjusted intermediate tensors and quantized adjusted second tensors.

17. The computer-implemented method of claim 1, wherein at least one of the steps of computing, processing, adjusting, and updating are performed on a server or in a data center and the output is streamed to a user device.

18. The computer-implemented method of claim 1, wherein at least one of the steps of computing, processing, adjusting, and updating are performed within a cloud computing environment.

19. The computer-implemented method of claim 1, wherein at least one of the steps of computing, processing, adjusting, and updating are performed for training, testing, or certifying the neural network employed in a machine, robot, or autonomous vehicle.

20. The computer-implemented method of claim 1, wherein at least one of the steps of computing, processing, adjusting, and updating is performed on a virtual machine comprising a portion of a graphics processing unit.

21. A system, comprising:

a processor configured to implement a neural network model comprising multiple processing layers by:

computing first clipping scalars for quantizing first tensors of a first processing layer that is coupled between two processing layers of the multiple processing layers;

processing an input by the neural network model, according to quantized tensors that include the quantized first tensors, by each layer of the multiple layers in sequence to produce intermediate tensors and an output of the neural network model;

adjusting the first tensors based on a loss gradient; and

updating the first clipping scalars based on a mean squared error to reduce differences between the adjusted first tensors and quantized adjusted first tensors.

34

22. The system of claim 21, wherein the first clipping scalars are recursively computed within the first processing layer according to a Newton-Raphson algorithm.

23. A non-transitory computer-readable media storing computer instructions for quantizing tensors of a neural network model comprising multiple processing layers that, when executed by one or more processors, cause the one or more processors to perform the steps of:

computing first clipping scalars for quantizing first tensors of a first processing layer that is coupled between two processing layers of the multiple processing layers;

processing an input by the neural network model, according to quantized tensors that include the quantized first tensors, by each layer of the multiple layers in sequence to produce intermediate tensors and an output of the neural network model;

adjusting the first tensors based on a loss gradient; and

updating the first clipping scalars based on a mean squared error to reduce differences between the adjusted first tensors and quantized adjusted first tensors.

24. The non-transitory computer-readable media of claim 23, wherein the first clipping scalars are computed by minimizing a quantization mean squared error.

25. A computer-implemented method for reducing power consumption of a neural network model comprising multiple processing layers, comprising:

computing first clipping scalars for quantizing first tensors of a first processing layer of the multiple processing layers;

processing an input by the neural network model, according to quantized tensors that include the quantized first tensors, by each processing layer of the multiple processing layers in sequence, to produce an output of the neural network model by consuming a first amount of power;

adjusting the first tensors based on a loss gradient;

updating the first clipping scalars based on a mean squared error to reduce differences between the adjusted first tensors and quantized adjusted first tensors; and

quantizing the first tensors using the updated first clipping scalars to produce second quantized first tensors, wherein processing the input by the neural network model according to the second quantized first tensors to produce a second output of the neural network model consumes a second amount of power that is less than the first amount of power.

26. The computer-implemented method of claim 25, wherein the first tensors are at least one of weights or activations.

* * * * *